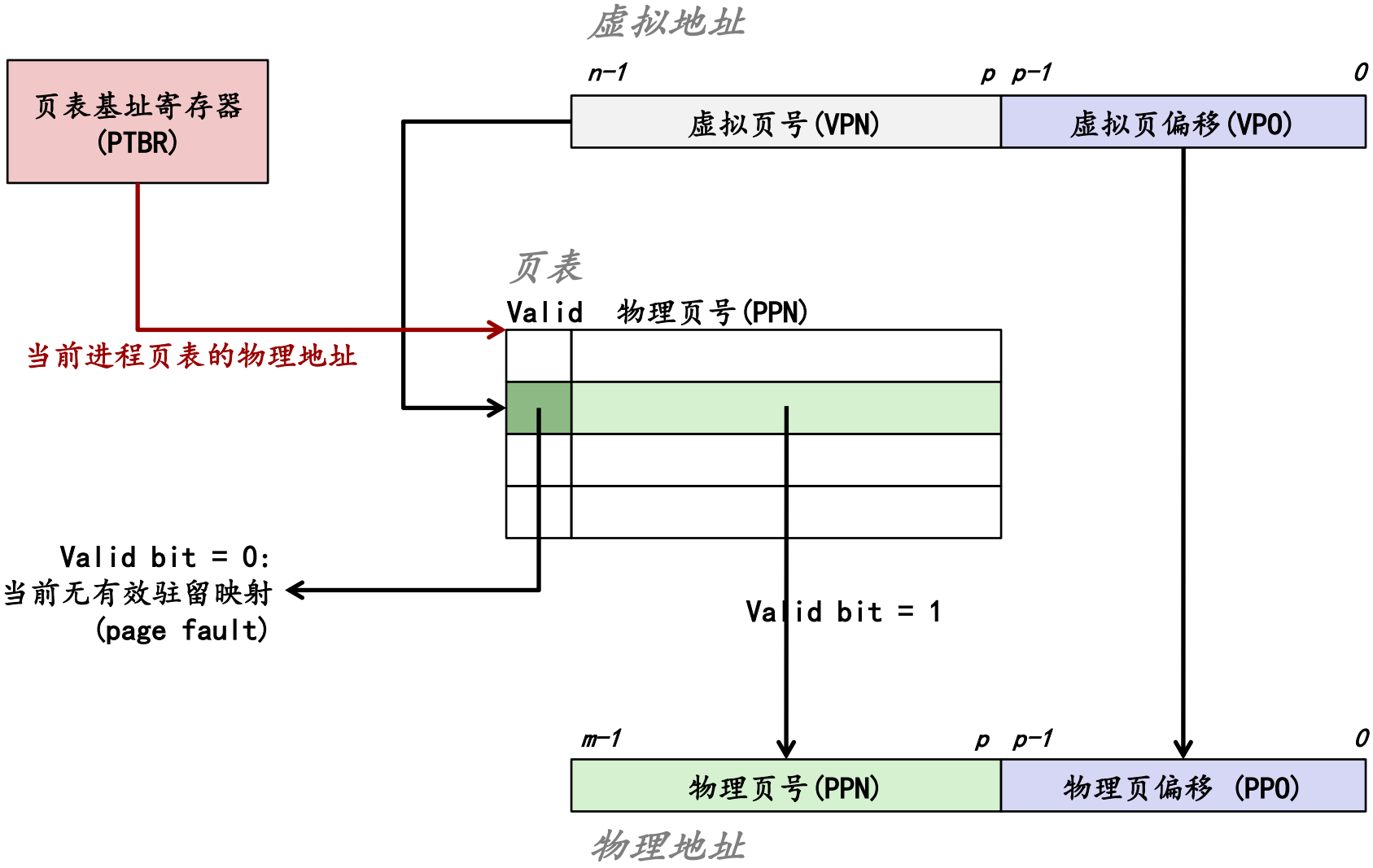
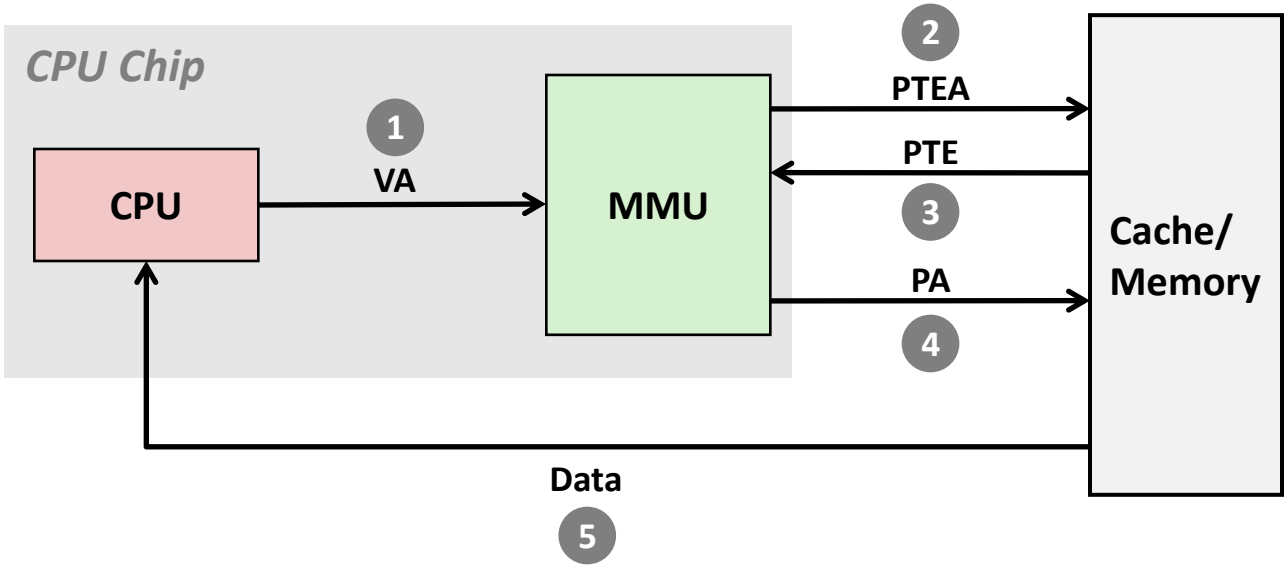
回顾：地址翻译符号总结

- 基本参数（Basic Parameters）
  - $N = 2^n$：虚拟地址空间中可寻址的地址数量
  - $M = 2^m$：物理地址空间中可寻址的地址数量
  - $P = 2^p$ ：页面大小(bytes)
- 虚拟地址（VA, Virtual Address）的组成
  - TLBI（TLB Index）：TLB 索引
  - TLBT（TLB Tag）：TLB 标签
  - VPO（Virtual Page Offset）：虚拟页偏移
  - VPN（Virtual Page Number）：虚拟页号
- 物理地址（PA, Physical Address）的组成
  - PPO（Physical Page Offset）：物理页偏移，与虚拟页偏移 VPO 完全相同
  - PPN（Physical Page Number）：物理页号

回顾：通过页表进行地址翻译

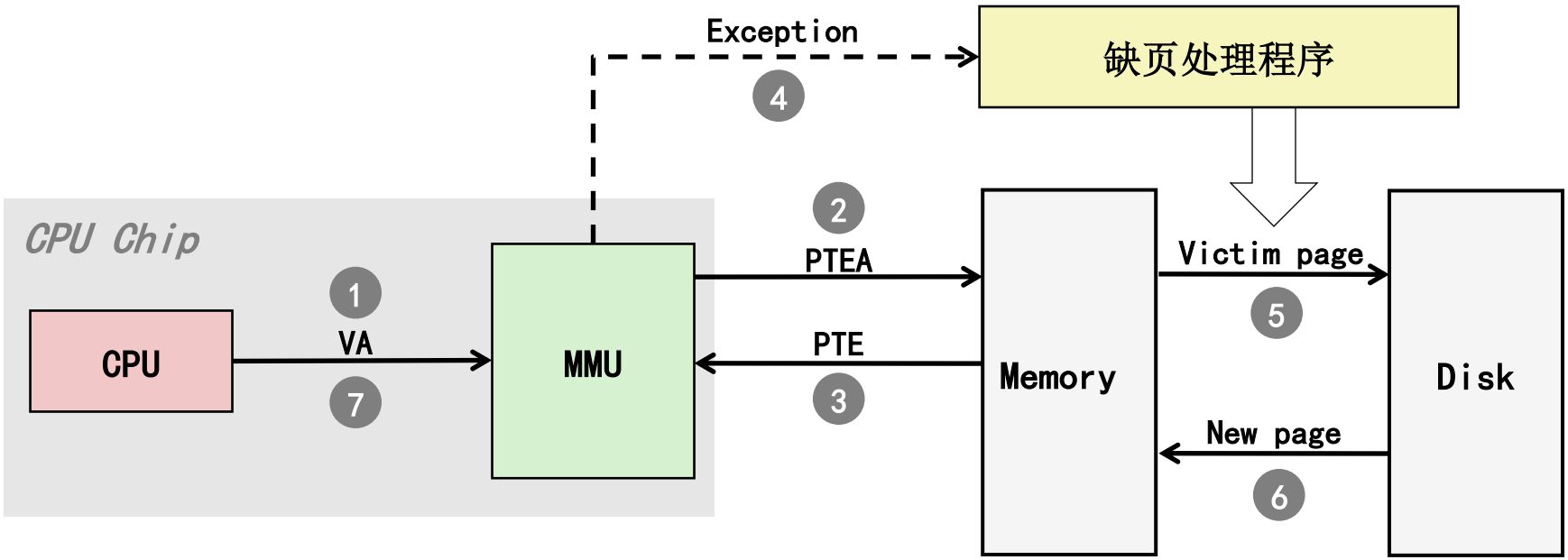


回顾：地址翻译：页命中（暂不考虑TLB）



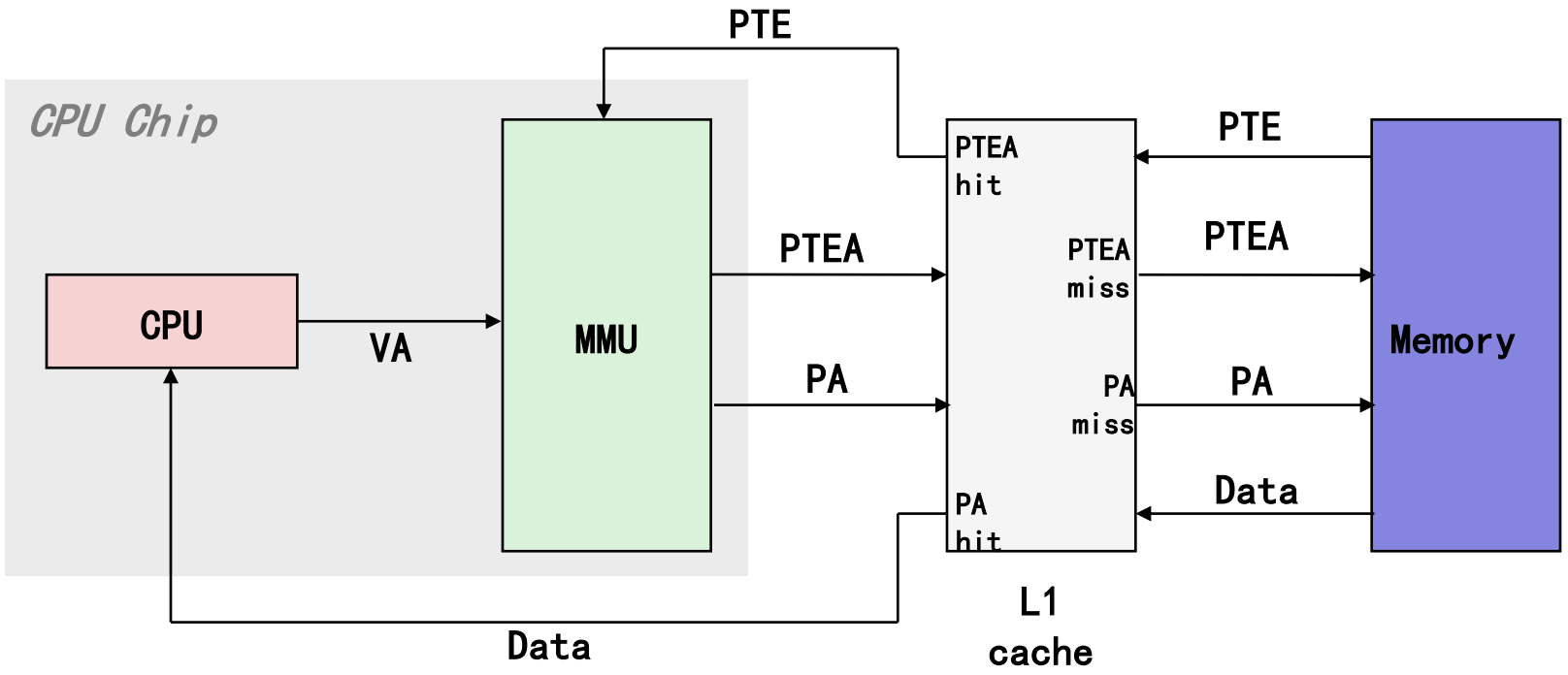
1. CPU将虚拟地址（VA）交给MMU进行翻译。
2. MMU 根据虚拟地址中的虚拟页号，到内存中的页表查找对应的页表项地址（PTEA）。
3. MMU 取出页表项（PTE），得到对应的物理页号。
4. MMU 将拼接后的物理地址（PA）发送到缓存/内存。
5. 缓存/内存根据物理地址找到数据，并把数据返回给处理器。

回顾：地址翻译：磁盘调页示例



1. 处理器将虚拟地址（VA）发送给 MMU（内存管理单元）。
2. MMU 访问页表项地址（PTEA）
3. MMU 根据虚拟页号，从内存中的页表中获取页表项（PTE）。
4. Valid Bit=0，MMU 会触发缺页异常（Page Fault Exception）。
5. 无空闲页时选择victim；需保留的脏内容写回后备存储
6. 缺页处理程序读取新页面，同时更新页表项（PTE）。
7. 缺页处理完成后，处理器重新执行指令，继续运行程序。

回顾：虚拟内存与缓存（Cache）的集成



VA（Virtual Address）：虚拟地址
PA（Physical Address）：物理地址
PTE（Page Table Entry）：页表项
PTEA（PTE Address）：页表项的地址
L1 Cache：一级缓存

第一步：CPU → MMU，发送虚拟地址 VA。
第二步：MMU 根据 VA 提取虚拟页号（VPN），查找页表项（PTE）。如果 PTEA 命中缓存，直接获得 PTE；如果 PTEA 未命中，从主存加载 PTE。
第三步：得到物理地址 PA 后，查询 L1 缓存：如果 PA 命中缓存 → 直接返回数据；如果 PA 未命中 → 去主存取数据，同时把数据写回缓存。第四步：将最终数据返回给 CPU。

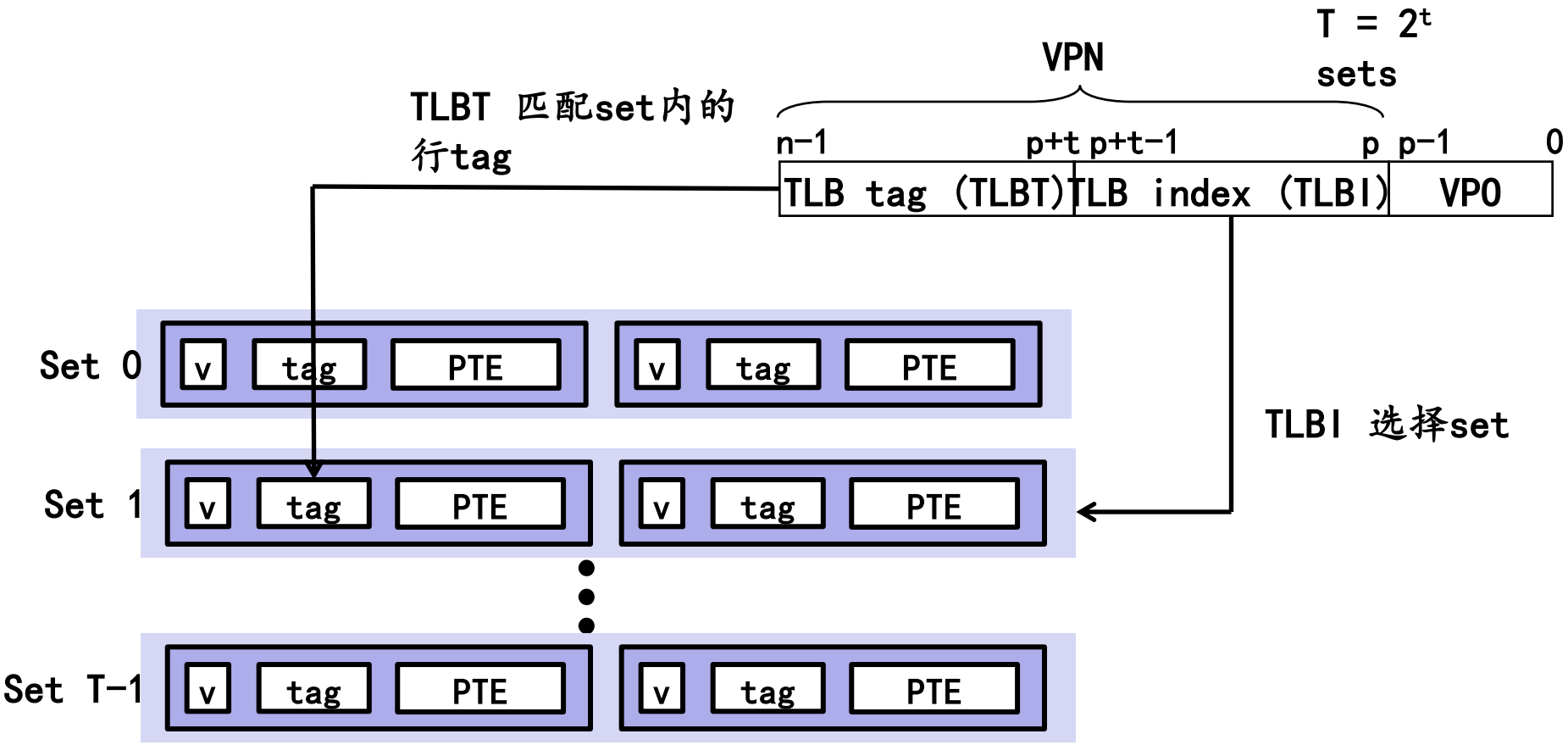
回顾：通过TLB加速地址转换

- 页表项（PTE，Page Table Entry）和普通数据一样，被缓存在L1 Cache中
  - PTE可能被逐出，如果L1缓存被其他数据占用
  - 即便PTE命中L1缓存仍然需要一次L1访问延迟

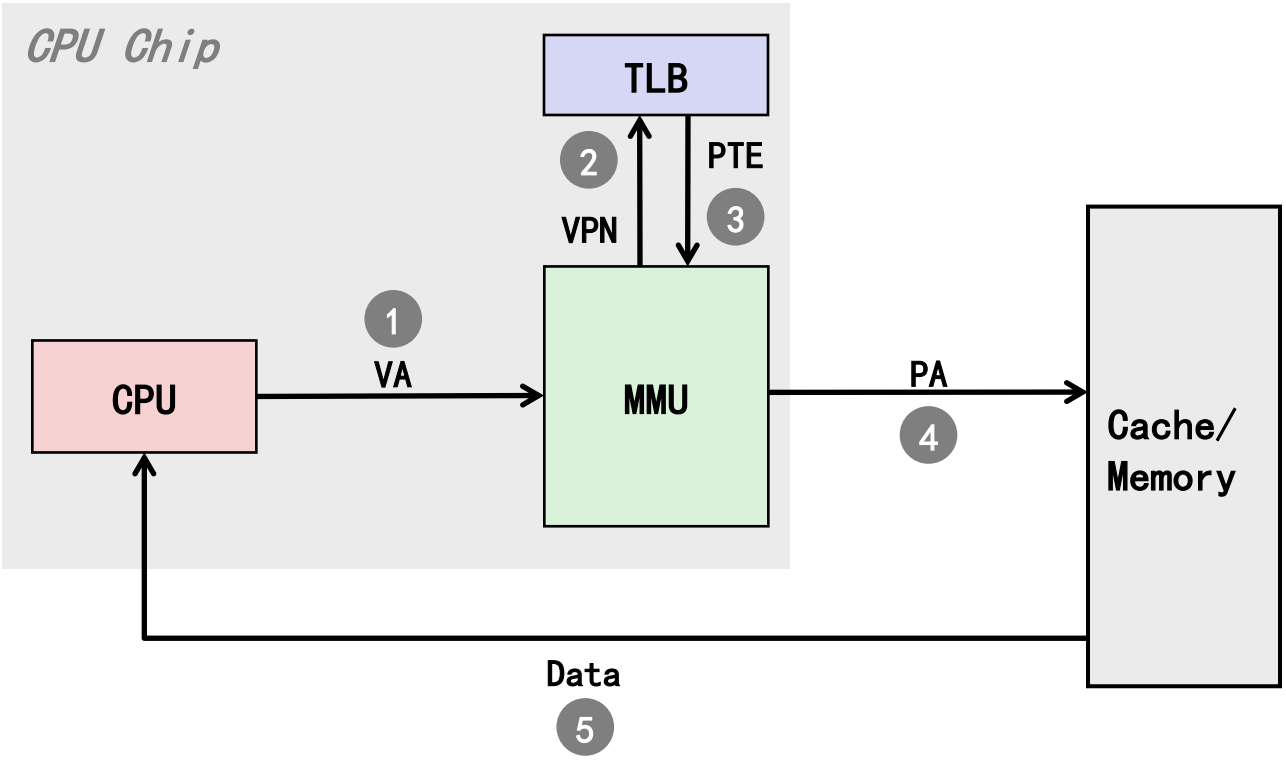
- 解决方案：*Translation Lookaside Buffer*（TLB）
  - TLB是一种专门用于加速虚拟地址到物理地址映射的小型硬件缓存，特点是：
    - TLB通常设计在MMU（内存管理单元）中，采用组相联（set-associative）结构，比L1缓存更高效。
    - 直接存储映射关系，它直接把虚拟页号（VPN）映射到物理页号（PPN），无需查找内存中的页表。
    - 存储少量完整页表项
      虽然TLB的容量小，但它能缓存最常用的一部分页表项，从而极大提高转换速度。

回顾：访问TLB

- MMU（内存管理单元）使用**虚拟地址（VA）中的虚拟页号（VPN）**部分来访问TLB：

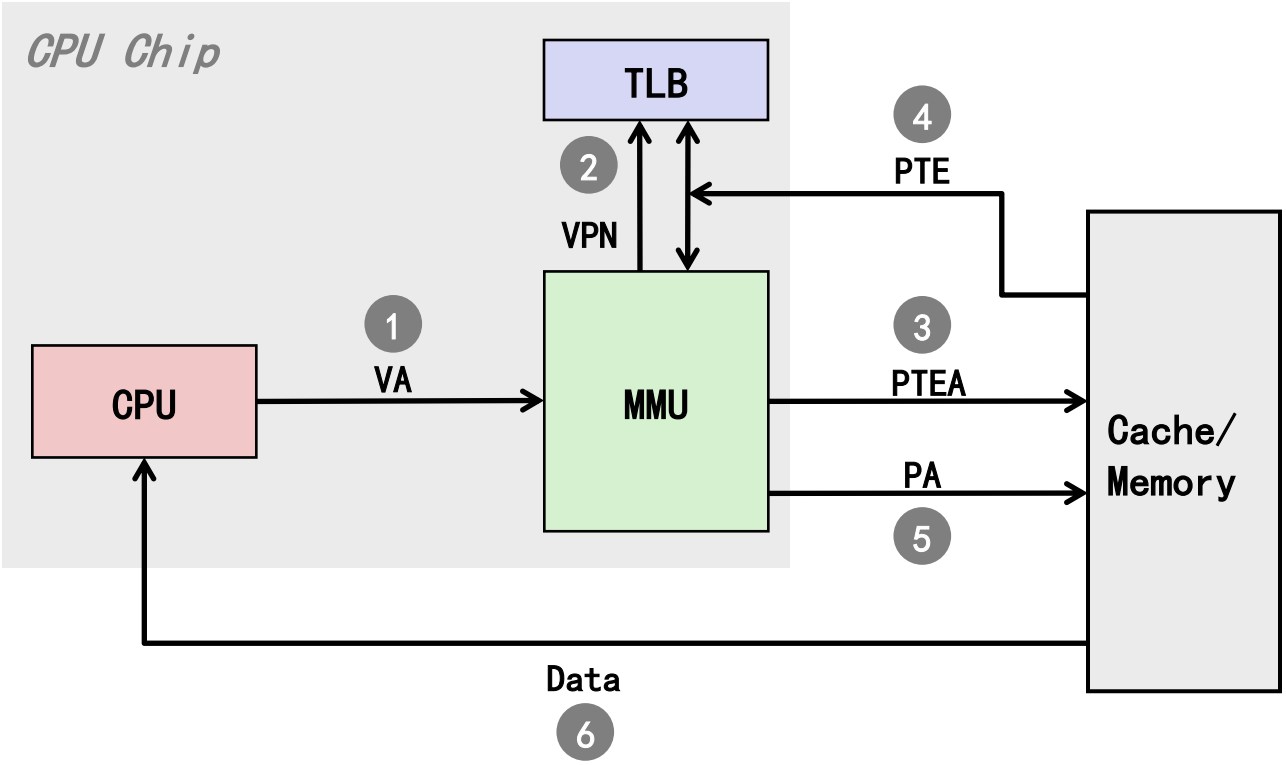


回顾：TLB 命中



TLB 命中省去额外的页表访问

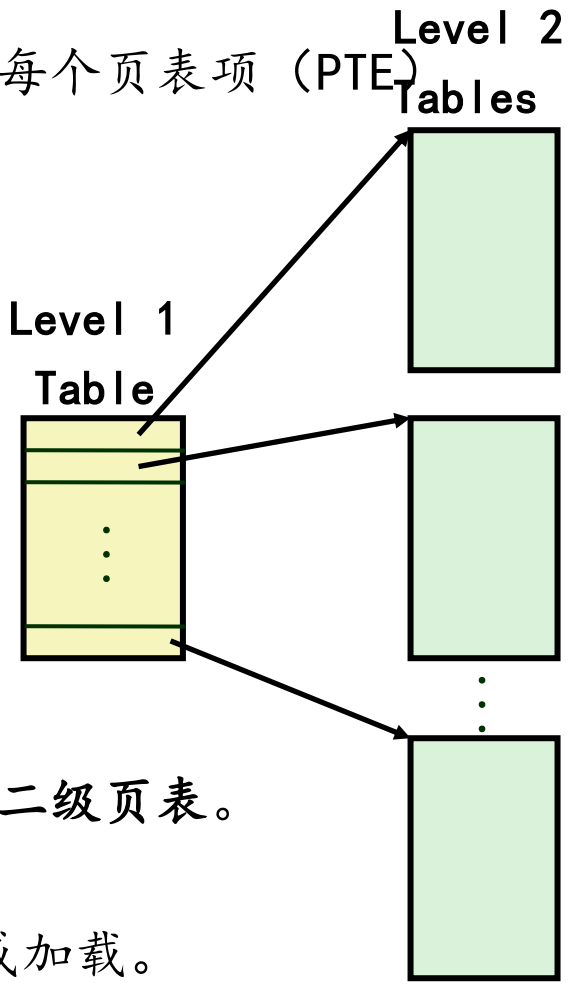
回顾： TLB Miss



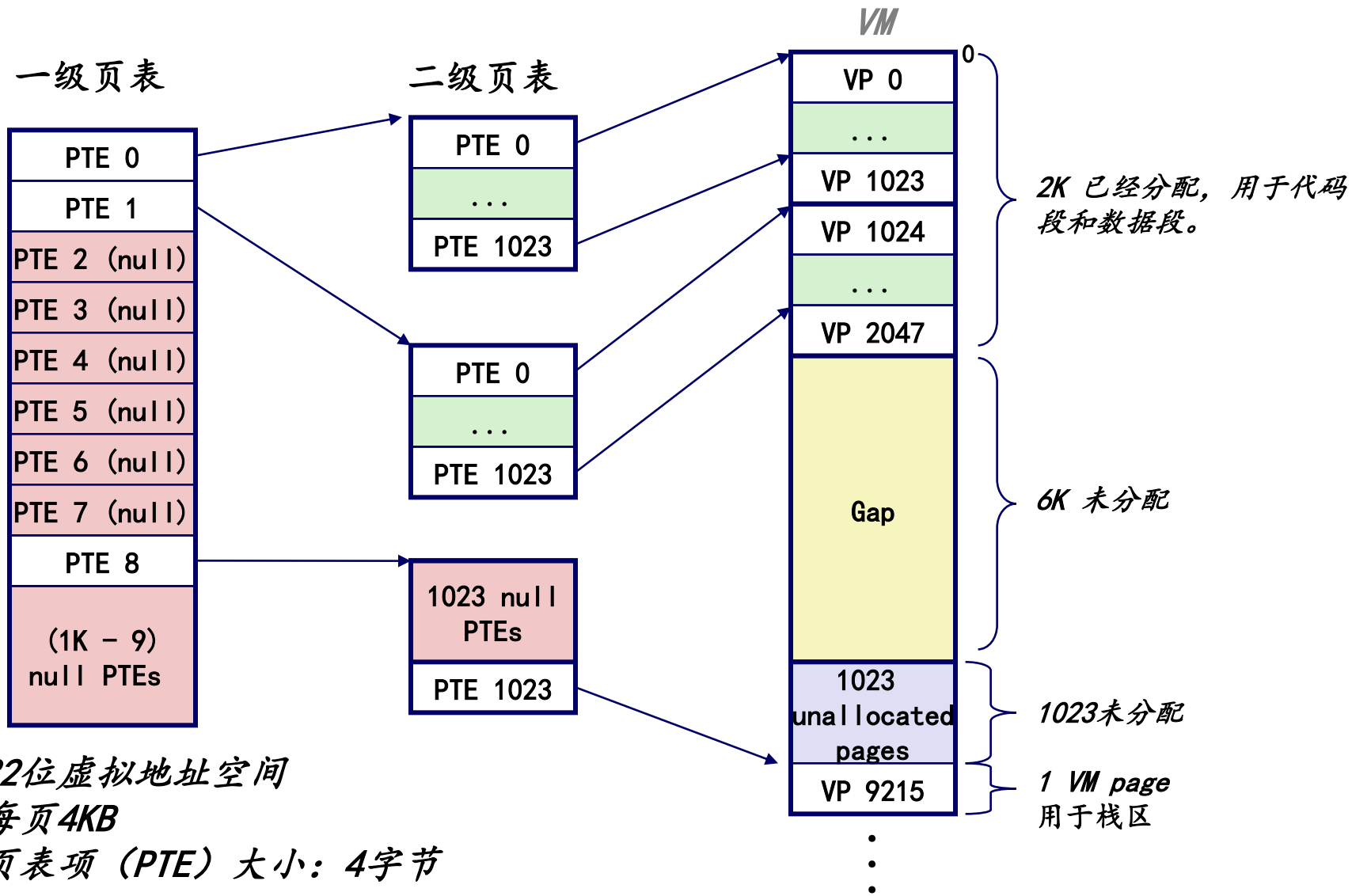
TLB 未命中需要额外查页表（可由缓存加速）。

多级页表（Multi-Level Page Tables）

- **假设场景**
  - 页大小：4KB（2^{12}字节），虚拟地址空间：48位，每个页表项（PTE）大小：8字节
- **问题：**
  - 单级页表需要 512 GiB 内存！
    - $2^{48} * 2^{-12} * 2^{3} = 2^{39}$ bytes
- **常见解决方案：多级页表**
- **两级页表示例**
  - 第一级页表（Level 1 Table）
    - 存放在内存中，**始终常驻**。
    - 每一项PTE不直接指向物理页面，而是指向**第二级页表**。
  - 第二级页表（Level 2 Table）
    - 只有当对应虚拟页范围被访问时，才会创建或加载。
    - 每一项PTE指向**具体的物理页**。
    - 未使用的地址范围无需创建下级页表。

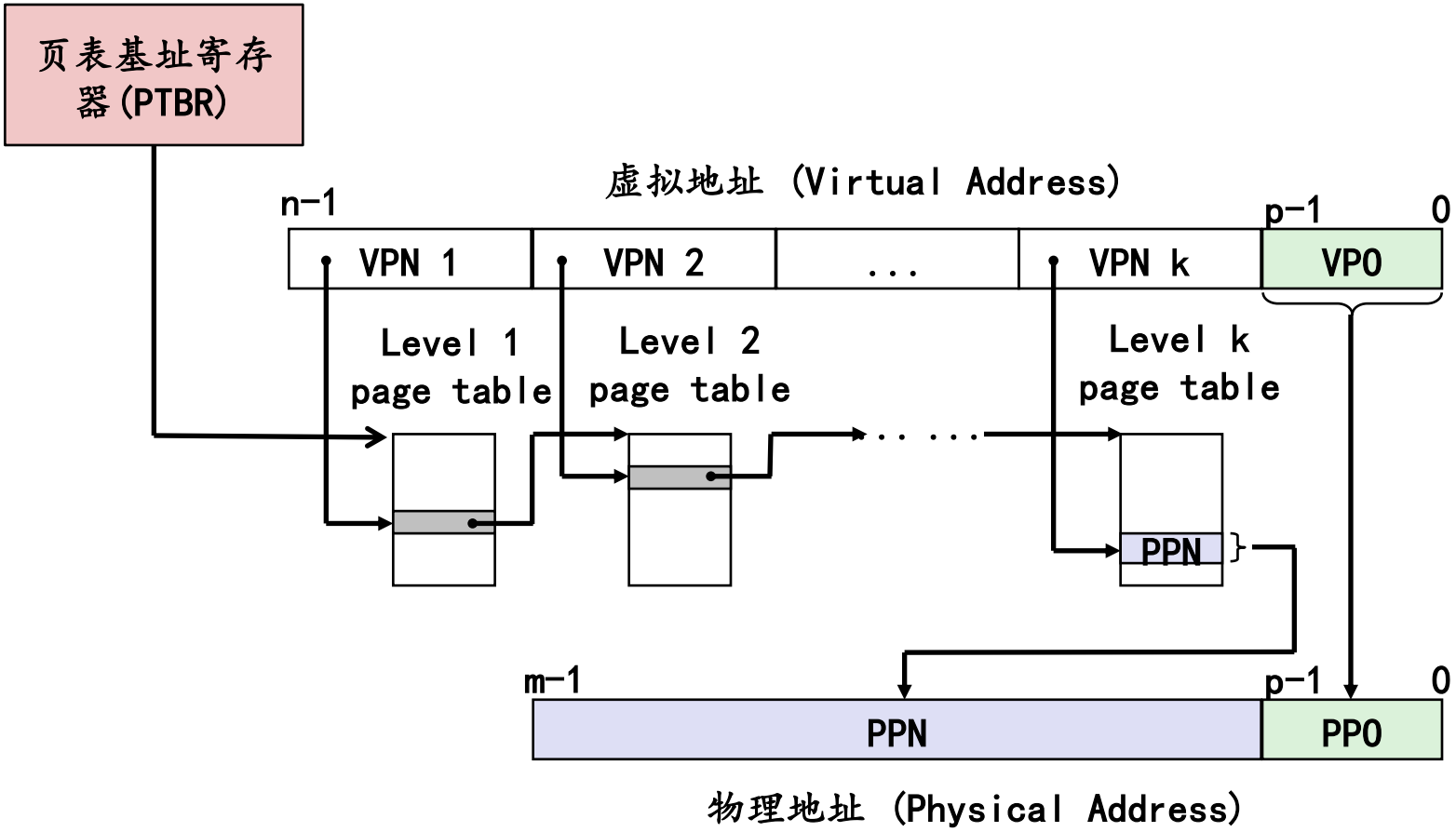


两级页表层次结构



- *32位虚拟地址空间*
- *每页4KB*
- *页表项（PTE）大小：4字节*

使用 k 级页表进行地址转换



总结

- **1. 程序员视角下的虚拟内存（Programmer’s view of virtual memory）**
  - 每个进程拥有自己独立的线性地址空间
    → 程序使用统一线性地址；其中可有未映射空洞。
  - 进程地址空间相互隔离
    共享区域需明确映射与权限。
- **2. 系统视角下的虚拟内存（System view of virtual memory）**
  - 高效利用内存
    - 系统通过**缓存虚拟内存页面**来减少频繁的磁盘访问。
    - 效率高的原因是**局部性原理**（locality）：程序大多数时间只访问一小部分数据。
  - 简化内存管理和编程
    - 操作系统帮程序员管理物理内存和地址映射，程序员不需要直接操心。
  - 简化内存保护
    - 虚拟内存提供了方便的检查点，操作系统能在这里验证访问权限，从而保护数据安全。

虚拟内存：系统

主要内容

- 简单内存系统示例
- 案例研究：Core i7/Linux 内存系统
- Linux内存映射

符号回顾

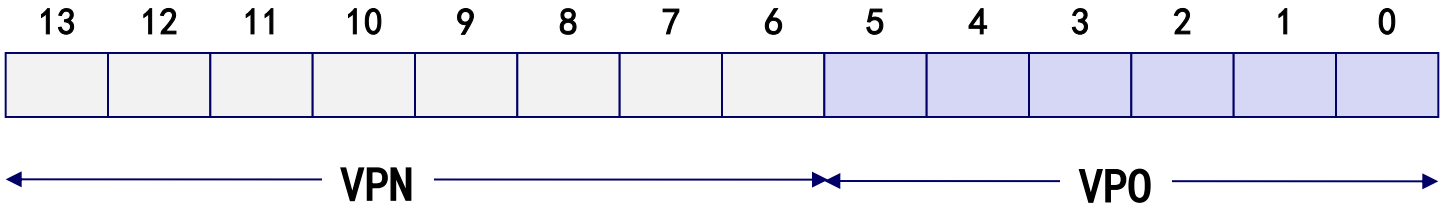
- 基本参数（Basic Parameters）
  - $N = 2^n$：虚拟地址空间中可寻址的地址数量
  - $M = 2^m$：物理地址空间中可寻址的地址数量
  - $P = 2^p$ ：页面大小(bytes)
- 虚拟地址（VA, Virtual Address）的组成
  - TLBI（TLB Index）：TLB 索引
  - TLBT（TLB Tag）：TLB 标签
  - VPO（Virtual Page Offset）：虚拟页偏移
  - VPN（Virtual Page Number）：虚拟页号
- 物理地址（PA, Physical Address）的组成
  - **PPO**（Physical Page Offset）：物理页偏移，与虚拟页偏移 VPO 完全相同
  - **PPN**（Physical Page Number）：物理页号
  - **CO**: Byte offset within cache line，偏移
  - **CI**: Cache 索引 index
  - **CT**: Cache 标记 tag

端到端的地址翻译：一个简单的例子

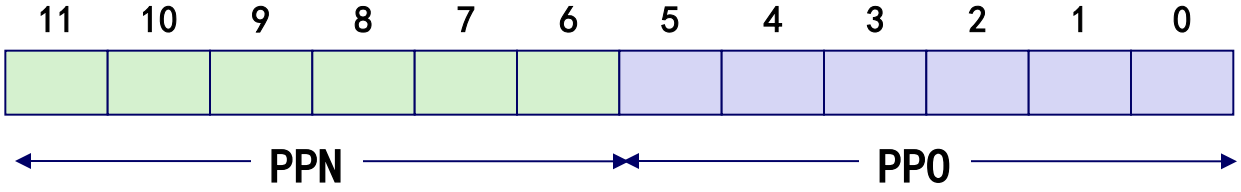
- 示例运行在一个有TLB和L1 d-cache的小系统上，为了简化，假设：
  - 存储器是按字节寻址的。
  - 存储器访问是针对1字节的字的
  - 虚拟地址是14位长的(n=14)。
  - 物理地址是12位长的(m=12)。
  - 页面大小是64字节(P=64)。
  - TLB是四路组相联的，总共有16个条目
  - L1 d-cache是物理寻址、直接映射的，行大小为4字节，总共有16组。

端到端的地址翻译:一个简单的例子

- 这里展示了虚拟地址和物理地址的格式。
- 因为每个页面是$2^6 = 64$字节，所以虚拟地址和物理地址的低6位分别作为VPO和PPO。虚拟地址的高8位作为VPN。物理地址的高6位作为PPN。



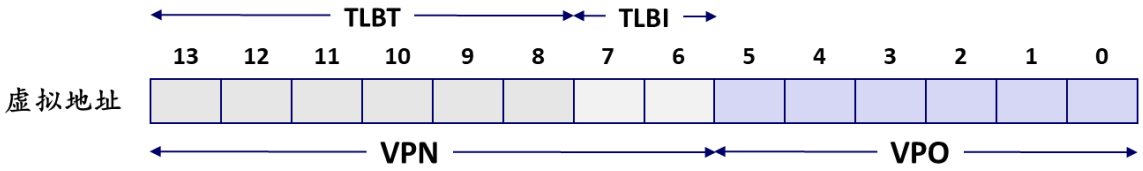
虚拟页号 Virtual Page Number　　虚拟页偏移 Virtual Page Offset



物理页号 Physical Page Number　物理页偏移 Physical Page Offset

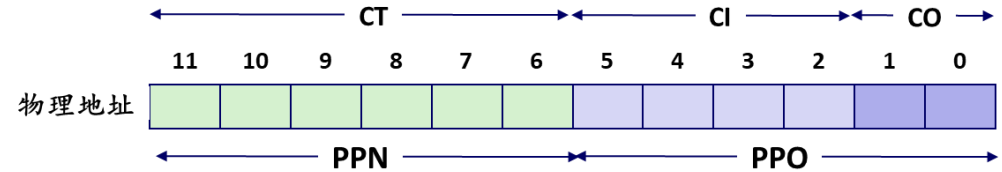
这个示例小存储器系统的一个快照

- 包括TLB（a）、页表的一部分（b），高速缓存（c）。
- 在TLB和高速缓存的图上面，我们还展示了访问这些设备时硬件是如何划分虚拟地址和物理地址的位的。



Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

a）TLB：四组，16个条目，四路组相连



Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	–	–	–	–
2	1B	1	00	02	04	08
3	36	0	–	–	–	–
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	–	–	–	–
7	16	1	11	C2	DF	03

Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	–	–	–	–
A	2D	1	93	15	DA	3B
B	0B	0	–	–	–	–
C	12	0	–	–	–	–
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	–	–	–	–

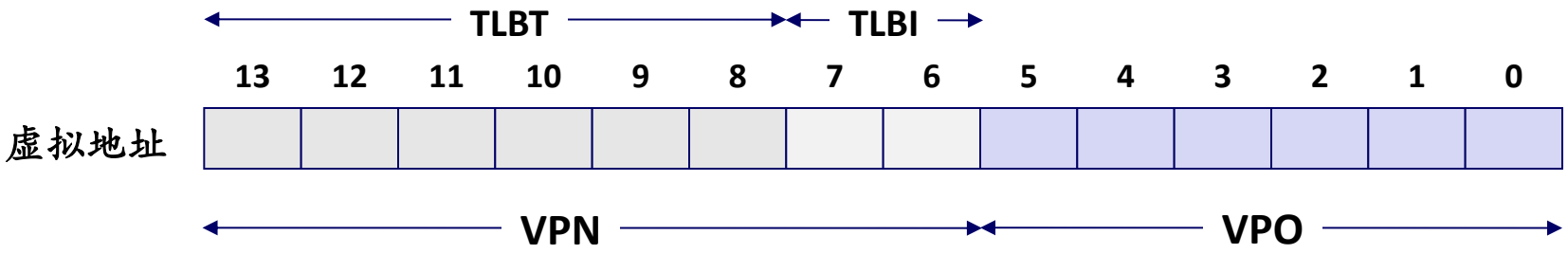
c）高速缓存：16个组，4字节的块，直接映射

VPN	PPN	Valid
00	28	1
01	–	0
02	33	1
03	02	1
04	–	0
05	16	1
06	–	0
07	–	0
08	13	1
09	17	1
0A	09	1
0B	–	0
0C	–	0
0D	2D	1
0E	11	1
0F	0D	1

b）页表，只展示了前16个PTE

1. 示例小内存系统中的TLB

- TLB是利用 VPN 的位进行虚拟寻址的。
- TLB有4个组，所以 VPN 的低2位就作为组索引(TLBI)，VPN中剩下的高6位作为标记(TLBT)，用来区别可能映射到同一个 TLB 组的不同的VPN。



Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

2.示例小内存系统中的页表

- 页表是一个单级设计，一共有256个页表条目(PTE)。只列出了开头16个。
- 每个无效PTE的PPN都用一个破折号来表示。

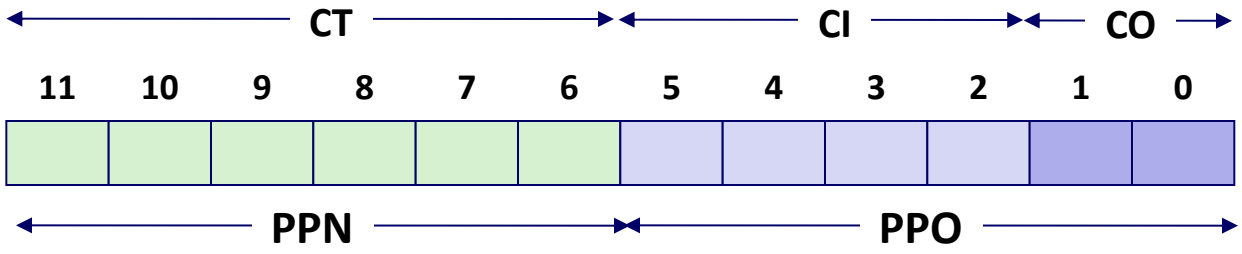
VPN	PPN	Valid
00	28	1
01	–	0
02	33	1
03	02	1
04	–	0
05	16	1
06	–	0
07	–	0

VPN	PPN	Valid
08	13	1
09	17	1
0A	09	1
0B	–	0
0C	–	0
0D	2D	1
0E	11	1
0F	0D	1

b) 页表，只展示了前16个PTE

3.示例小内存系统中的高速缓存(L1)

- 直接映射的缓存
- 通过物理地址中的字段来寻址
- 每个块都是4字节，所以物理地址底2位作为块偏移
- 一共16组，4位表示组索引（CI），6位表示标记（CT）



Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	–	–	–	–
2	1B	1	00	02	04	08
3	36	0	–	–	–	–
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	–	–	–	–
7	16	1	11	C2	DF	03

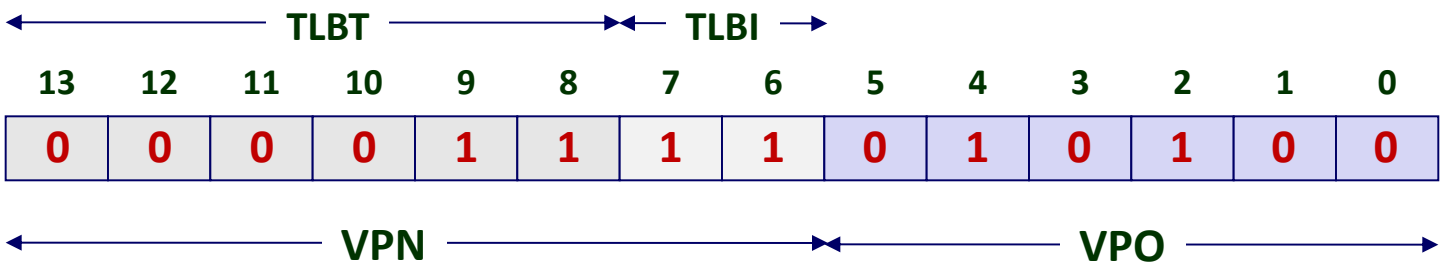
Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	–	–	–	–
A	2D	1	93	15	DA	3B
B	0B	0	–	–	–	–
C	12	0	–	–	–	–
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	–	–	–	–

地址翻译例子#1

虚拟地址：0x03D4

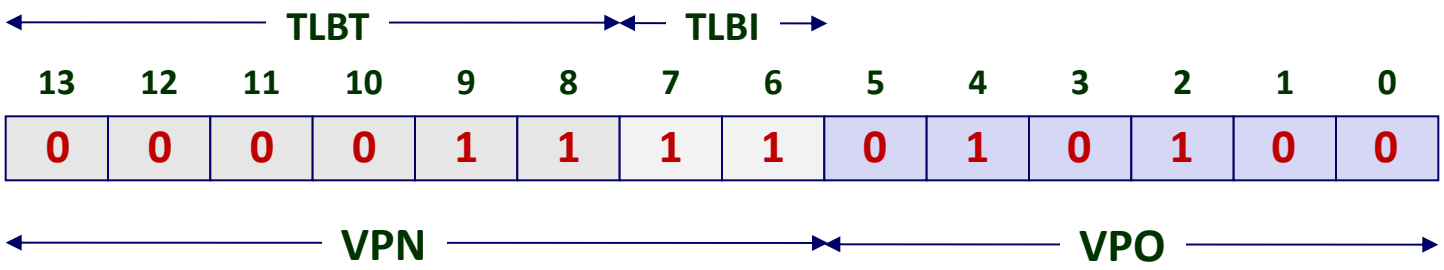
地址翻译例子#1

虚拟地址：0x03D4：



地址翻译例子#1

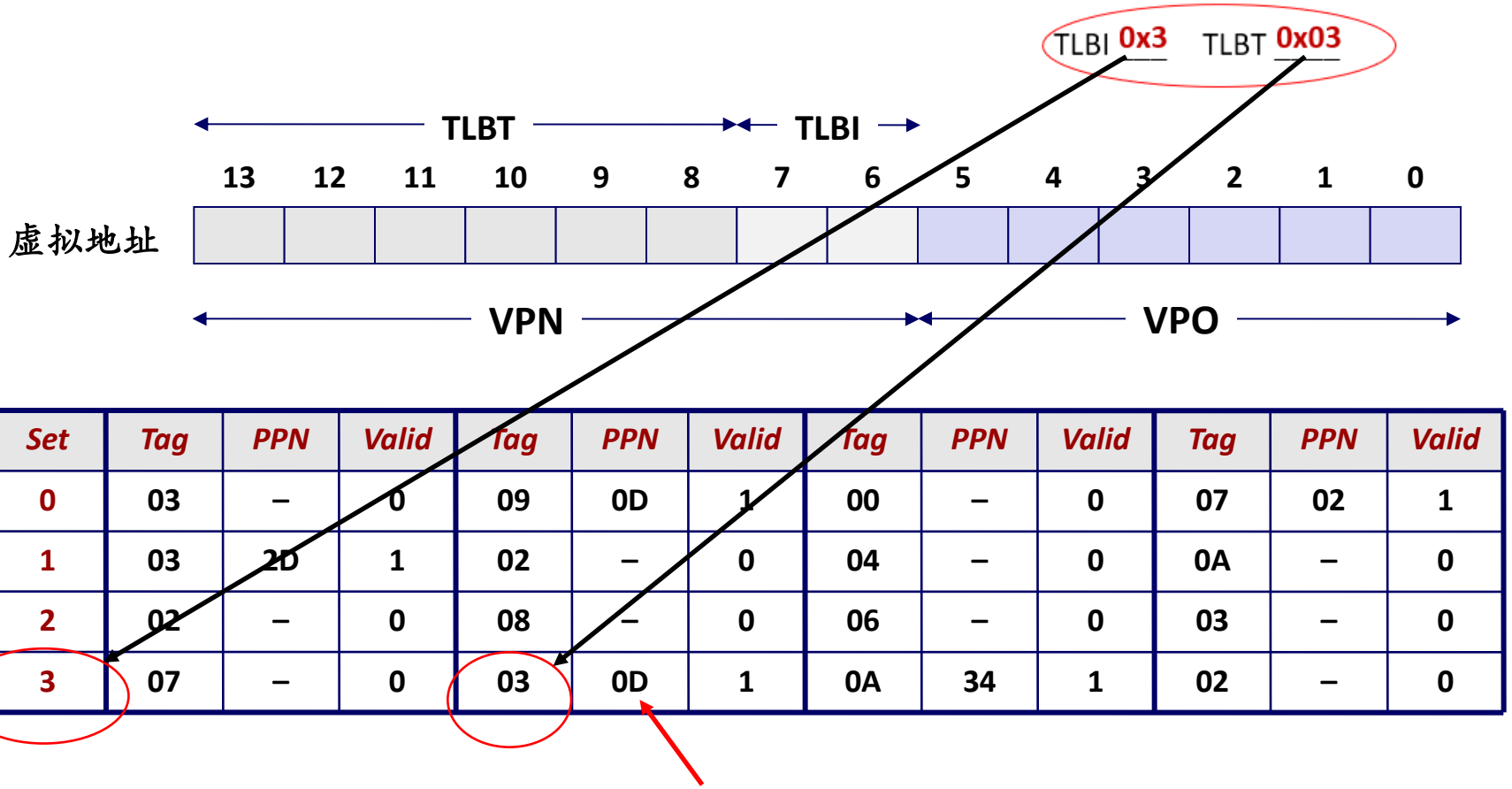
虚拟地址：0x03D4：



VPN 0x0F　　TLBI 0x03　TLBT 0x03　　TLB Hit? __　　Page Fault? __　　PPN: ____

1. 示例小内存系统中的TLB

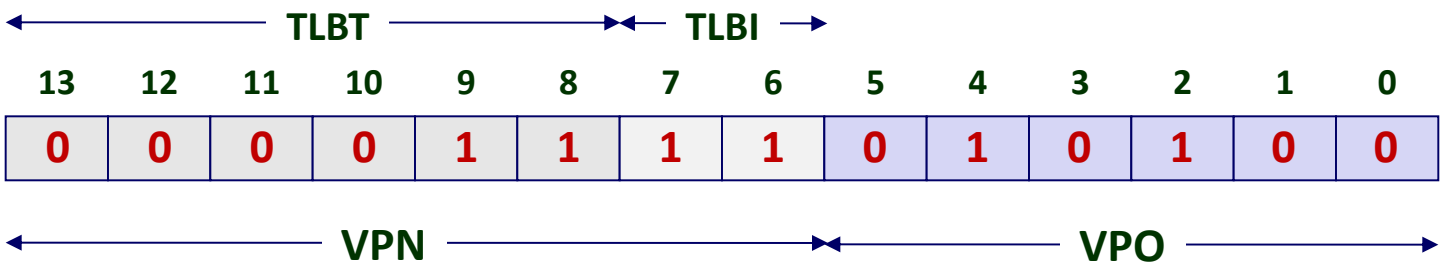
- TLB是利用 VPN 的位进行虚拟寻址的。
- TLB有4个组，所以 VPN 的低2位就作为组索引(TLBI)，VPN中剩下的高6位作为标记(TLBT)，用来区别可能映射到同一个 TLB 组的不同的VPN。



Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

地址翻译例子#1

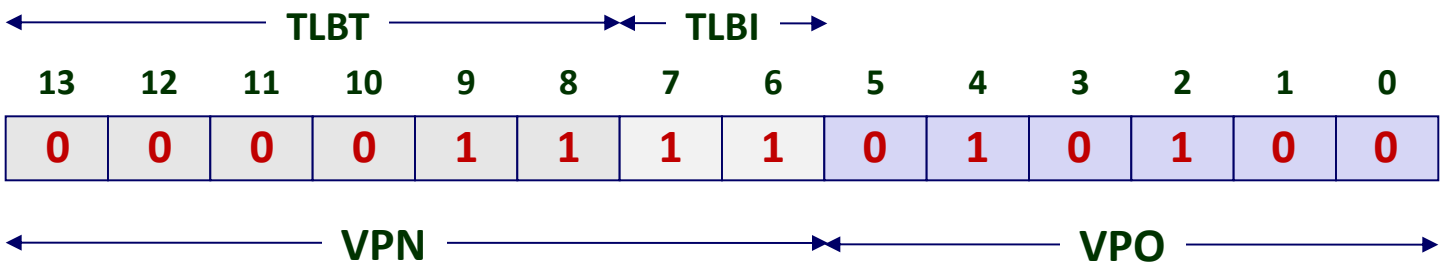
虚拟地址：0x03D4：



VPN 0x0F TLBI 0x03 TLBT 0x03 TLB Hit? Y Page Fault? N PPN: 0x0D

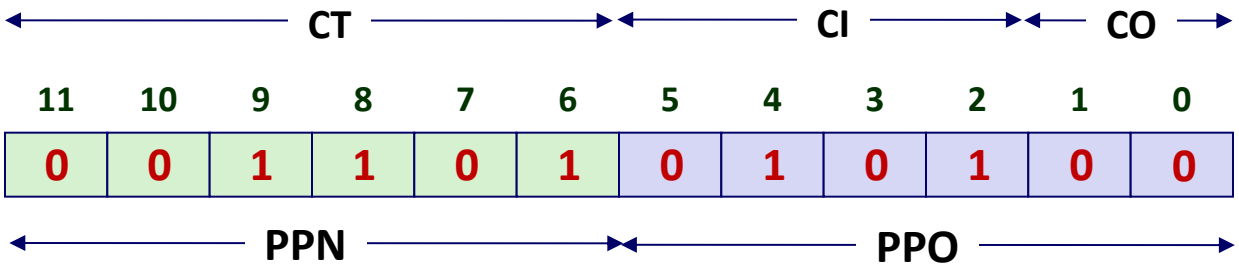
地址翻译例子#1

虚拟地址：0x03D4：



VPN 0x0F TLBI 0x03 TLBT 0x03 TLB Hit? Y Page Fault? N PPN: 0x0D

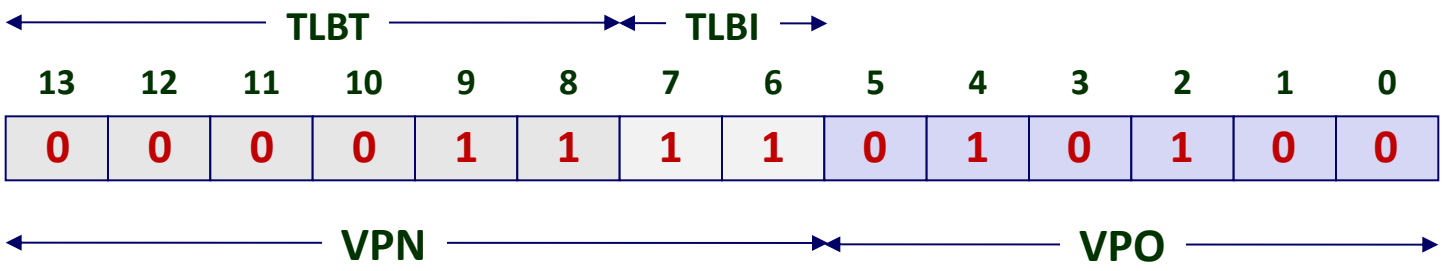
物理地址：0x354



CO ___ CI___ CT ____ Hit? __ Byte: ____

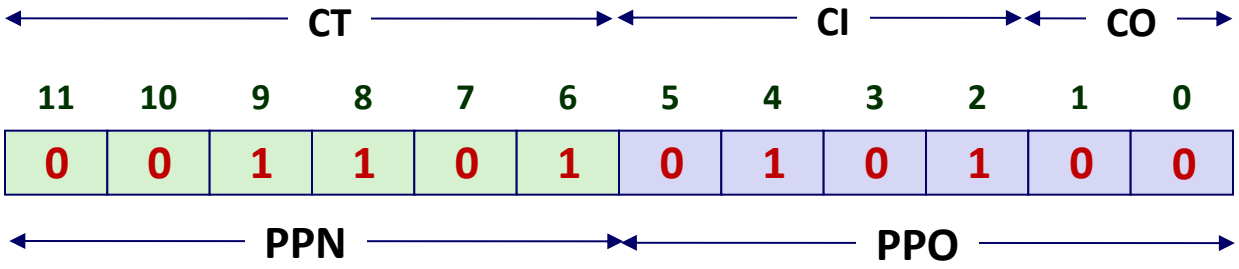
地址翻译例子#1

虚拟地址：0x03D4：



VPN 0x0F TLBI 0x03 TLBT 0x03 TLB Hit? Y Page Fault? N PPN: 0x0D

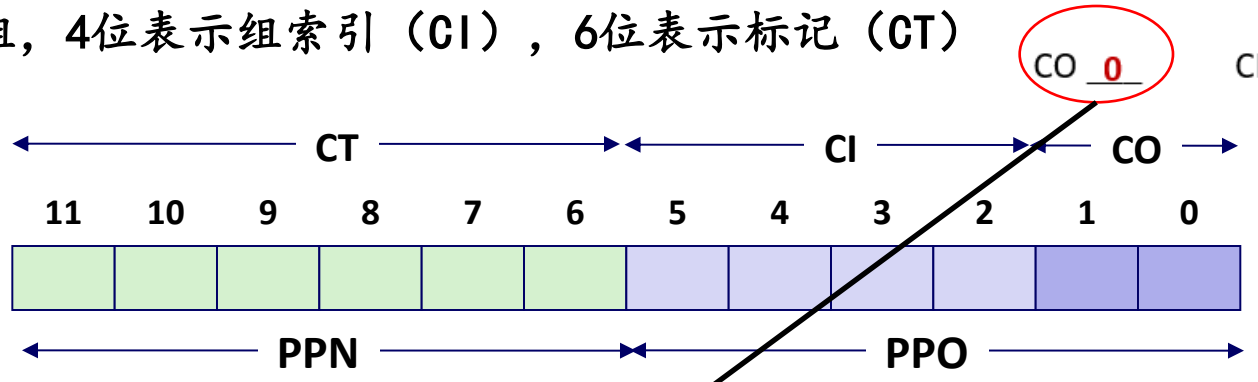
物理地址：



CO 0x0 CI 0x05 CT 0x0D Hit? __ Byte: ____

3. 示例的小内存系统中的高速缓存(L1)

- 直接映射的缓存
- 通过物理地址中的字段来寻址
- 每个块都是4字节，所以物理地址底2位作为块偏移
- 一共16组，4位表示组索引（CI），6位表示标记（CT）

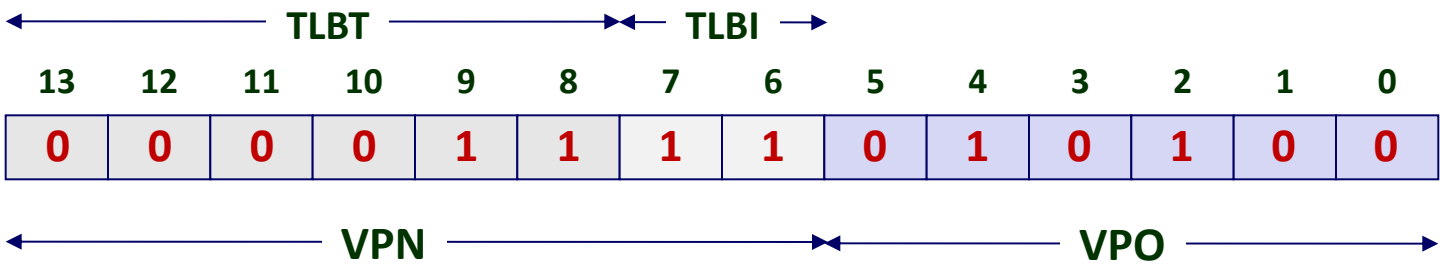


Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	–	–	–	–
2	1B	1	00	02	04	08
3	36	0	–	–	–	–
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	–	–	–	–
7	16	1	11	C2	DF	03

Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	–	–	–	–
A	2D	1	93	15	DA	3B
B	0B	0	–	–	–	–
C	12	0	–	–	–	–
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	–	–	–	–

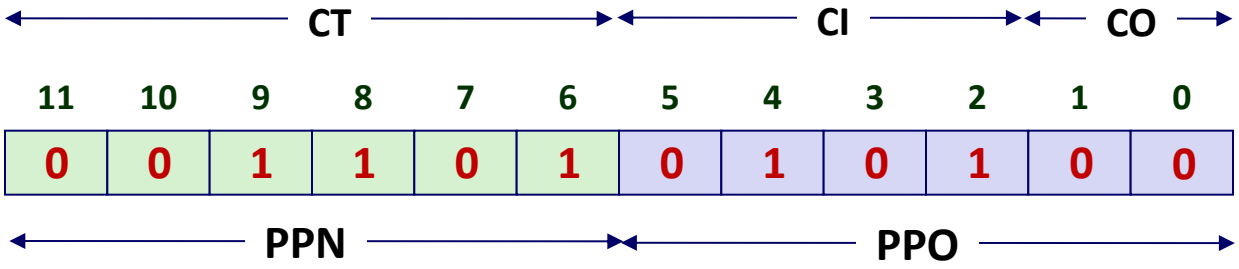
地址翻译例子#1

虚拟地址：0x03D4：



VPN 0x0F　　TLBI 0x3　　TLBT 0x03　　TLB Hit? Y　　Page Fault? N　　PPN: 0x0D

物理地址：



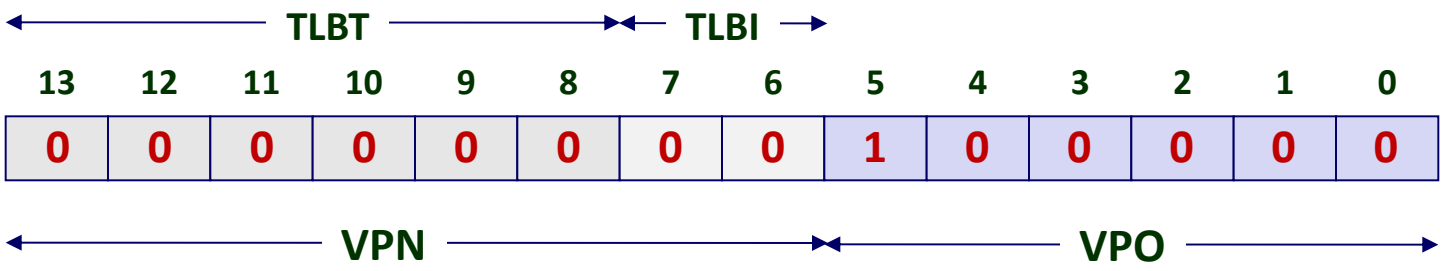
CO 0　　CI 0x5　　CT 0x0D　　Hit? Y　　Byte: 0x36

地址翻译例子#2

虚拟地址：0x0020

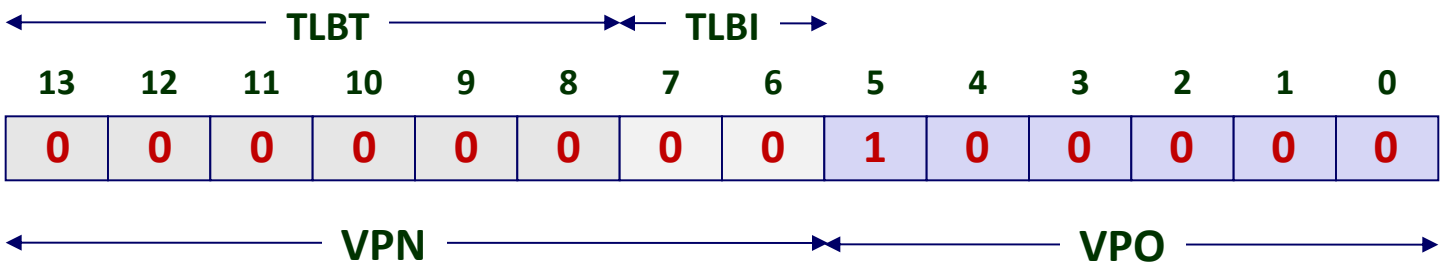
地址翻译例子#2

虚拟地址：0x0020



地址翻译例子#2

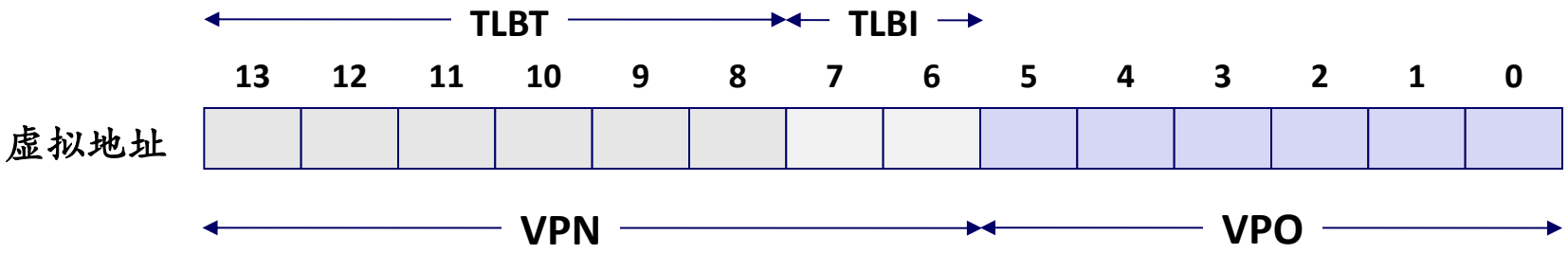
虚拟地址：0x0020



VPN 0x00 TLBI 0 TLBT 0x00 TLB Hit? __ Page Fault? __ PPN: ____

1. 示例小内存系统中的TLB

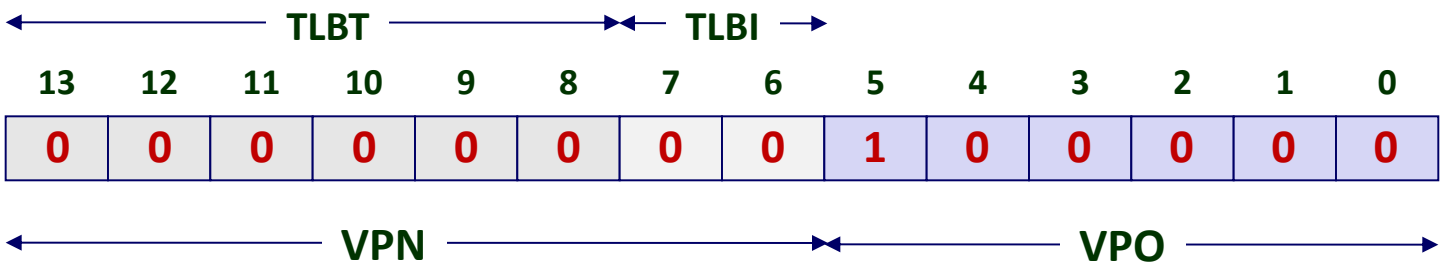
- TLB是利用 VPN 的位进行虚拟寻址的。
- TLB有4个组，所以 VPN 的低2位就作为组索引(TLBI)，VPN中剩下的高6位作为标记(TLBT)，用来区别可能映射到同一个 TLB 组的不同的VPN。



Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

地址翻译例子#2

虚拟地址：0x0020



VPN 0x00 TLBI 0 TLBT 0x00 TLB Hit? N Page Fault? __ PPN: ____

2.示例小内存系统中的页表

- 页表是一个单级设计，一共有256个页表条目(PTE)。只列出了开头16个。
- 每个无效PTE的PPN都用一个破折号来表示。

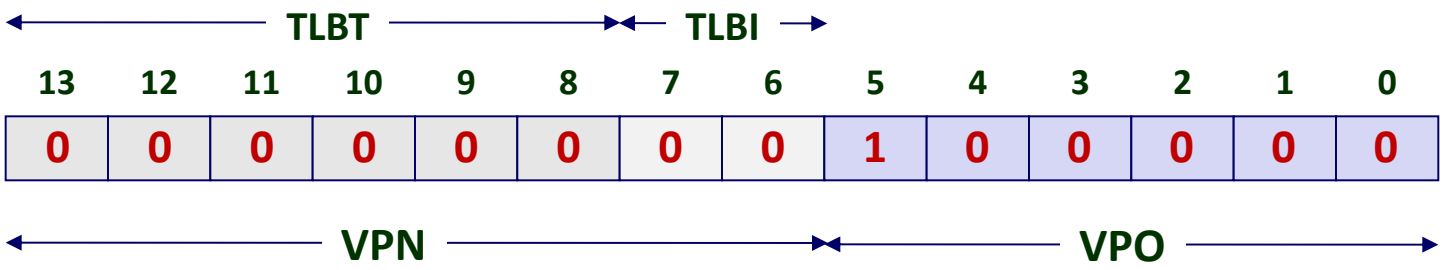
VPN	PPN	Valid
00	28	1
01	–	0
02	33	1
03	02	1
04	–	0
05	16	1
06	–	0
07	–	0

VPN	PPN	Valid
08	13	1
09	17	1
0A	09	1
0B	–	0
0C	–	0
0D	2D	1
0E	11	1
0F	0D	1

b) 页表，只展示了前16个PTE

地址翻译例子#2

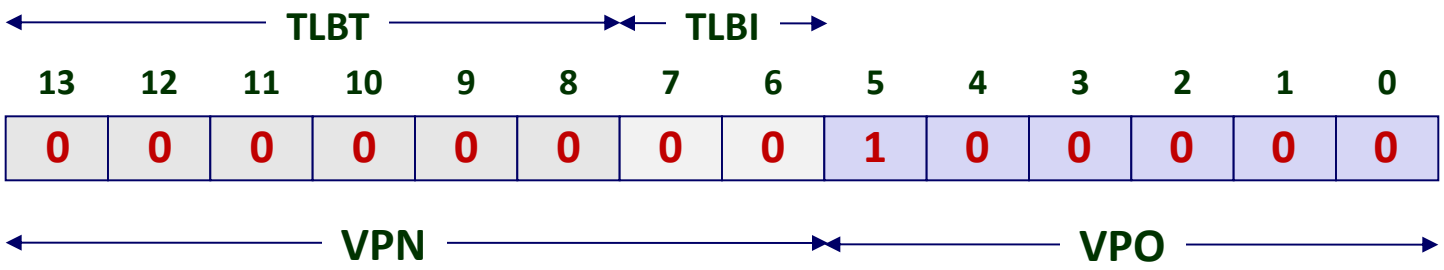
虚拟地址：0x0020



VPN 0x00 TLBI 0 TLBT 0x00 TLB Hit? N Page Fault? N PPN: 0x28

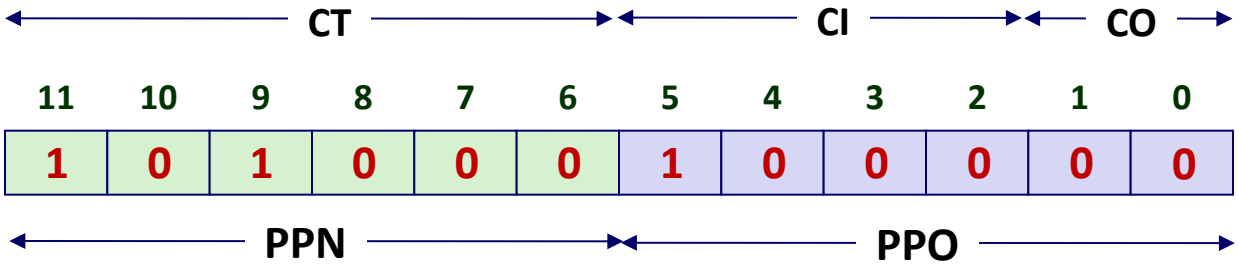
地址翻译例子#2

虚拟地址：0x0020



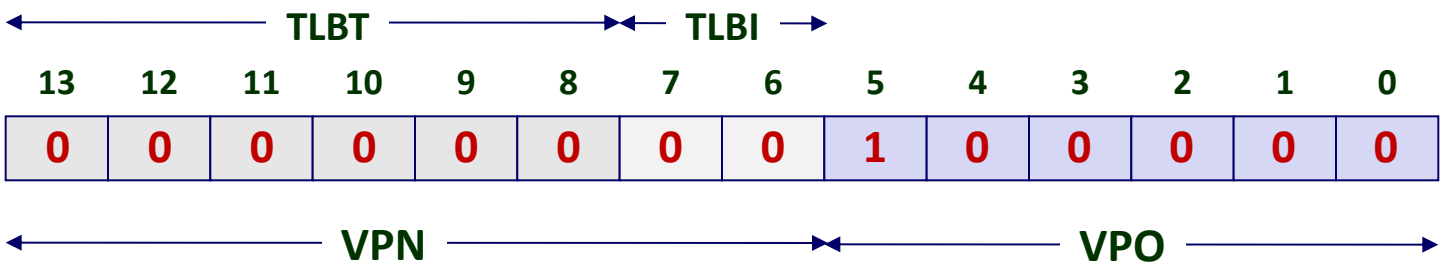
VPN 0x00 TLBI 0 TLBT 0x00 TLB Hit? N Page Fault? N PPN: 0x28

物理地址：



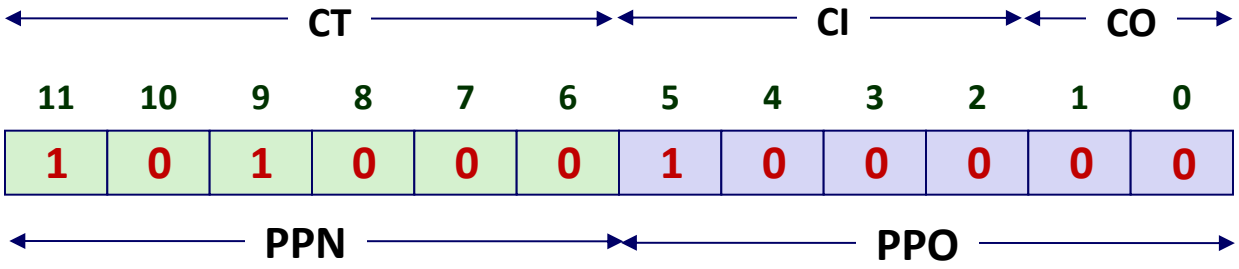
地址翻译例子#2

虚拟地址：0x0020



VPN 0x00 TLBI 0 TLBT 0x00 TLB Hit? N Page Fault? N PPN: 0x28

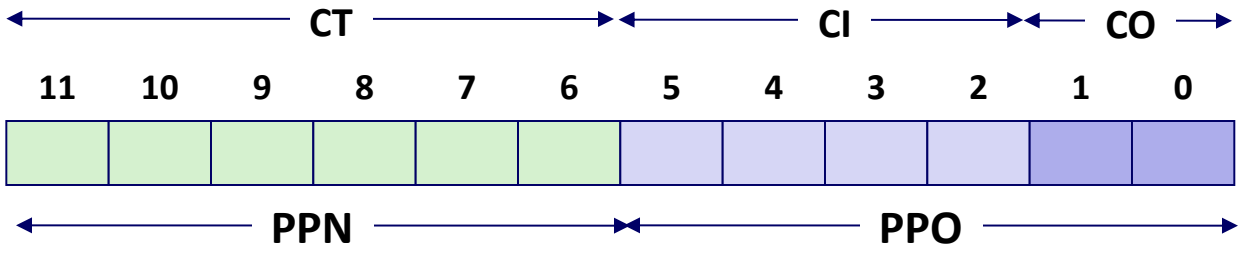
物理地址：



CO 0 CI 0x8 CT 0x28 Hit? __ Byte: ____

3. 示例的小内存系统中的高速缓存(L1)

- 直接映射的缓存
- 通过物理地址中的字段来寻址
- 每个块都是4字节，所以物理地址底2位作为块偏移
- 一共16组，4位表示组索引（CI），6位表示标记（CT）

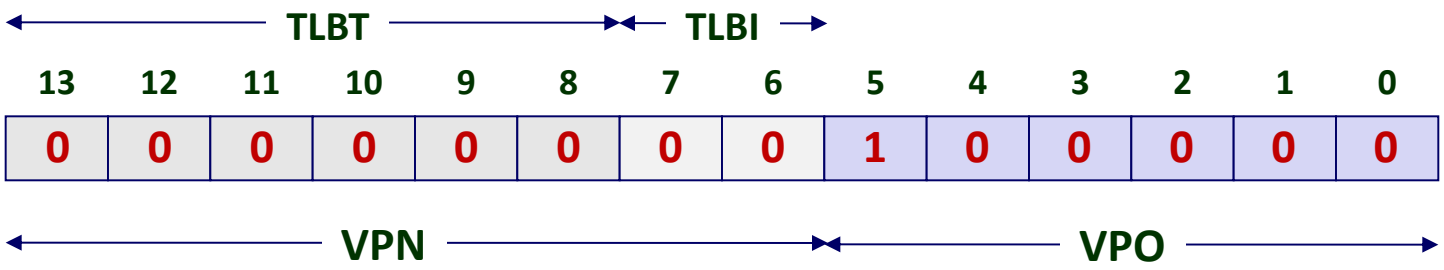


Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	–	–	–	–
2	1B	1	00	02	04	08
3	36	0	–	–	–	–
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	–	–	–	–
7	16	1	11	C2	DF	03

Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	–	–	–	–
A	2D	1	93	15	DA	3B
B	0B	0	–	–	–	–
C	12	0	–	–	–	–
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	–	–	–	–

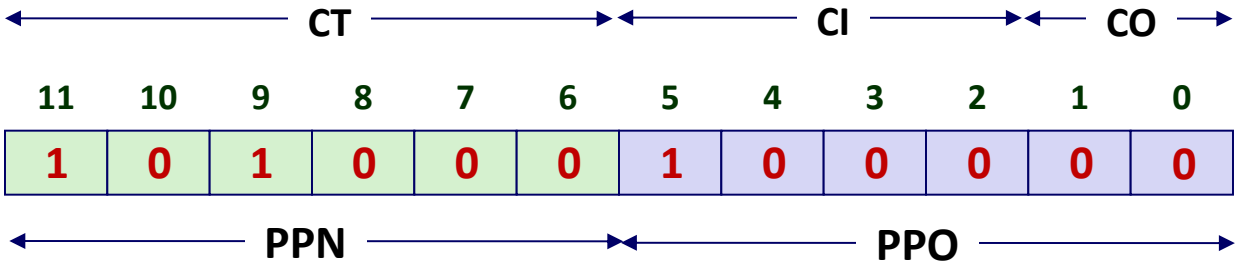
地址翻译例子#2

虚拟地址：0x0020



VPN 0x00 TLBI 0 TLBT 0x00 TLB Hit? N Page Fault? N PPN: 0x28

物理地址：

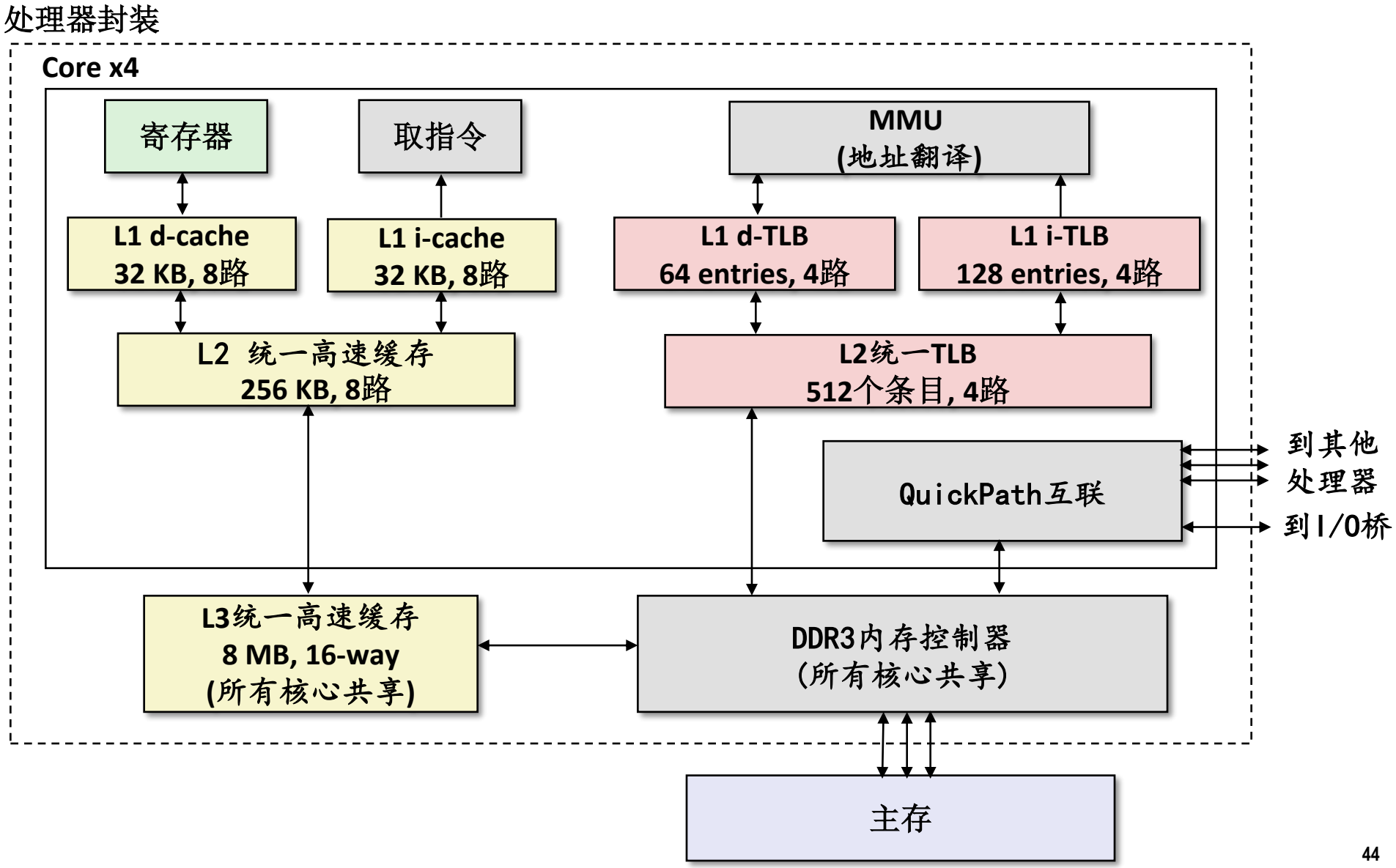


CO 0 CI 0x8 CT 0x28 Hit? N Byte: Mem

主要内容

- 简单内存系统示例
- 案例研究：Core i7/Linux 内存系统
- Linux内存映射

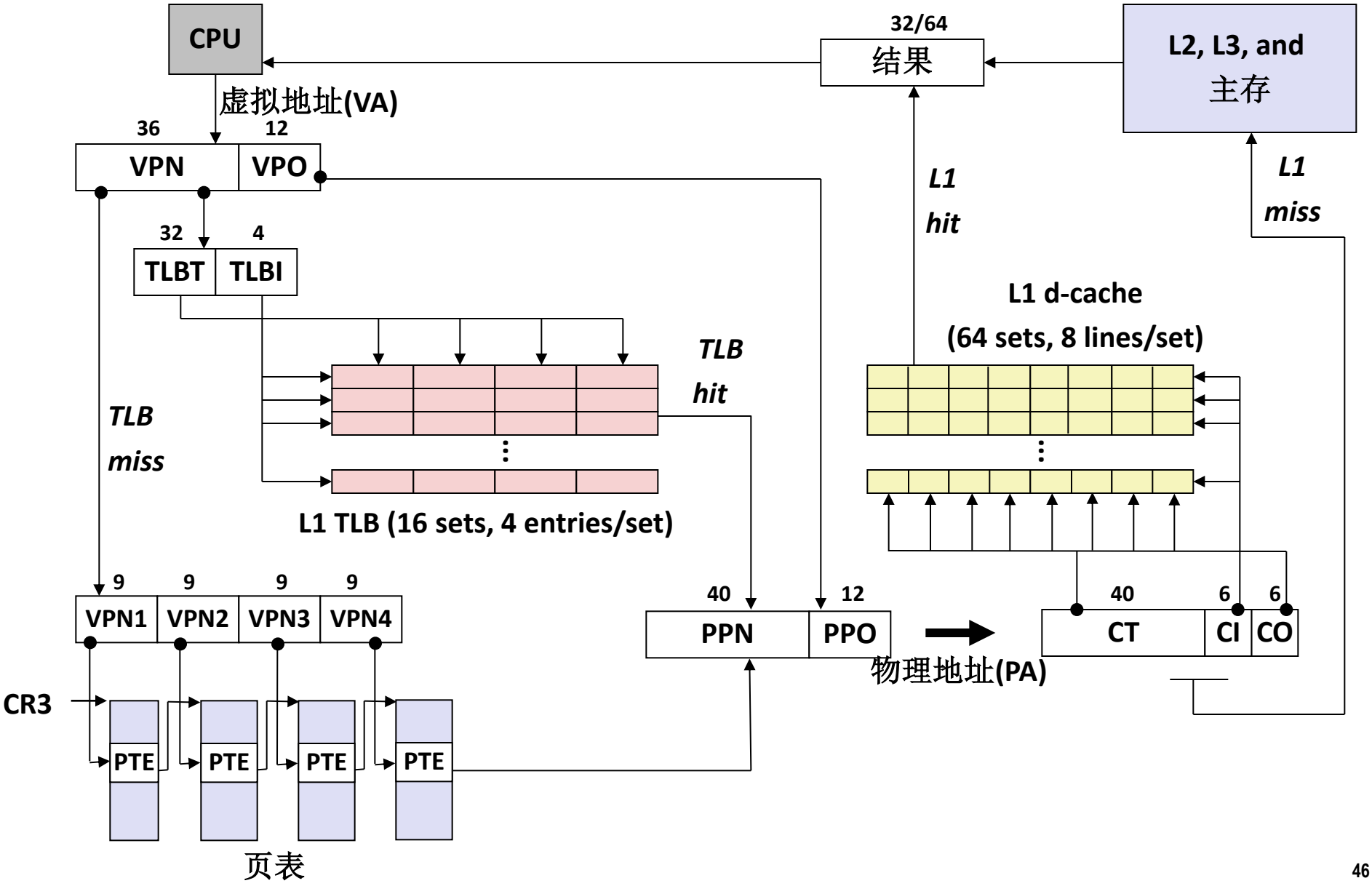
早期 Core i7 内存教学模型



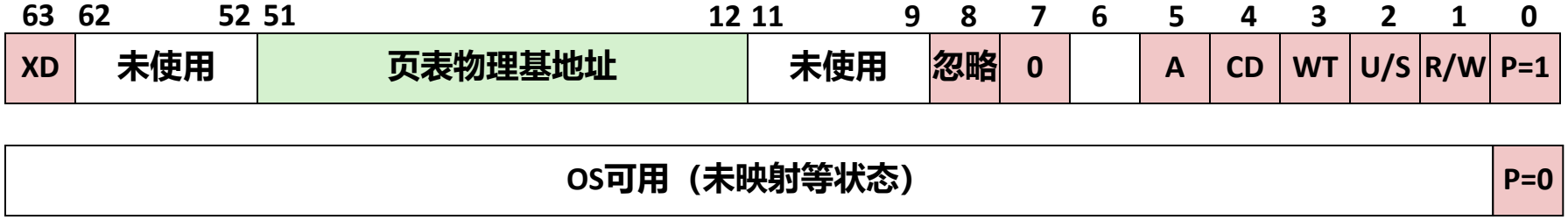
符号回顾

- 基本参数（Basic Parameters）
  - $N = 2^n$：虚拟地址空间中可寻址的地址数量
  - $M = 2^m$：物理地址空间中可寻址的地址数量
  - $P = 2^p$ ：页面大小(bytes)
- 虚拟地址（VA, Virtual Address）的组成
  - TLBI（TLB Index）：TLB 索引
  - TLBT（TLB Tag）：TLB 标签
  - VPO（Virtual Page Offset）：虚拟页偏移
  - VPN（Virtual Page Number）：虚拟页号
- 物理地址（PA, Physical Address）的组成
  - **PPO**（Physical Page Offset）：物理页偏移，与虚拟页偏移 VPO 完全相同
  - **PPN**（Physical Page Number）：物理页号
  - **CO**: Byte offset within cache line, 偏移
  - **CI**: Cache index
  - **CT**: Cache tag

端到端Core i7 地址翻译简化概况



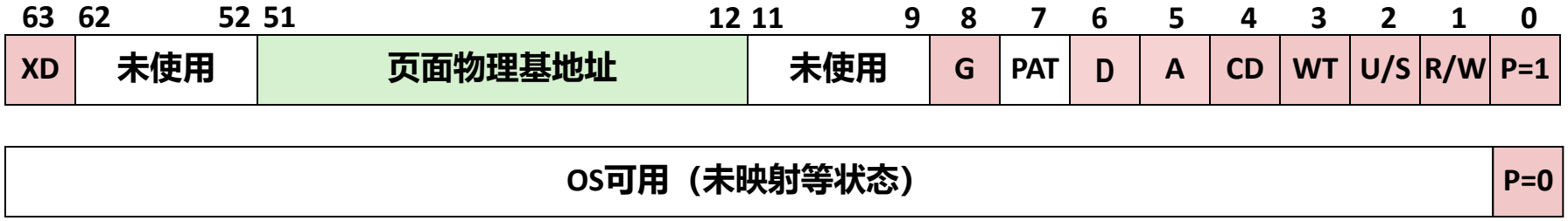
第1－3级非叶页表项（简化）



非叶项指向4KiB子页表。主要字段如下：

- P：1表示子页表在内存中；0表示当前条目不present。
- R/W：为所有可访问页面设置读写权限。只读或读写。
- U/S：为所有可访问页面设置用户模式或特权（内核）模式的访问权限。
- WT：与CD等共同决定访问子页表的缓存属性。
- A：访问位，由硬件在使用该条目时设置，软件可清零。
- 大页：第3级可映射2MiB；支持时第2级可映射1GiB。
- 页表基址：地址位51:12；实际宽度由处理器决定，4KiB对齐。
- XD：禁止或允许从该页表项可访问的所有页面中取指令。

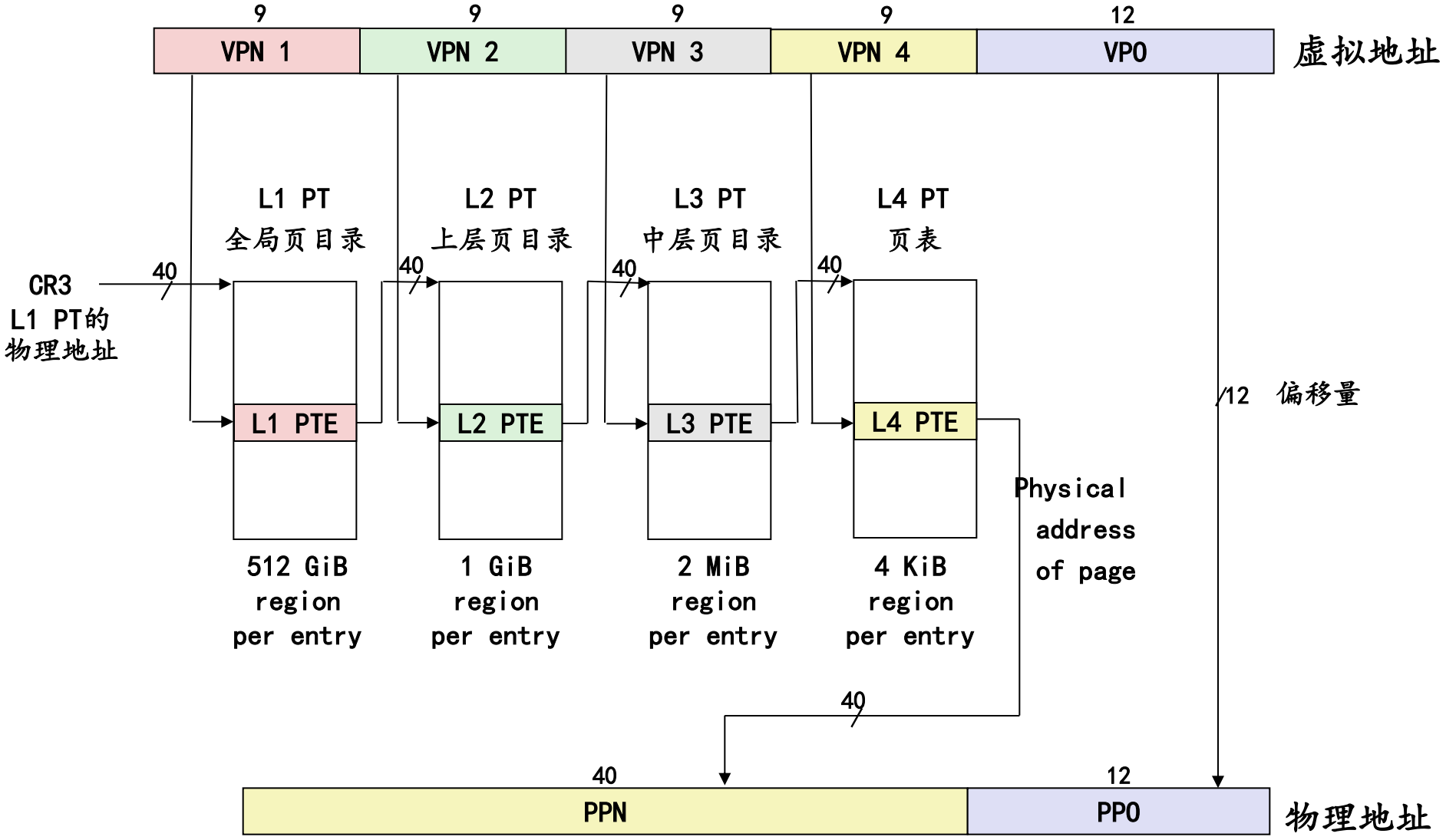
Corei7 第4级页表条目格式



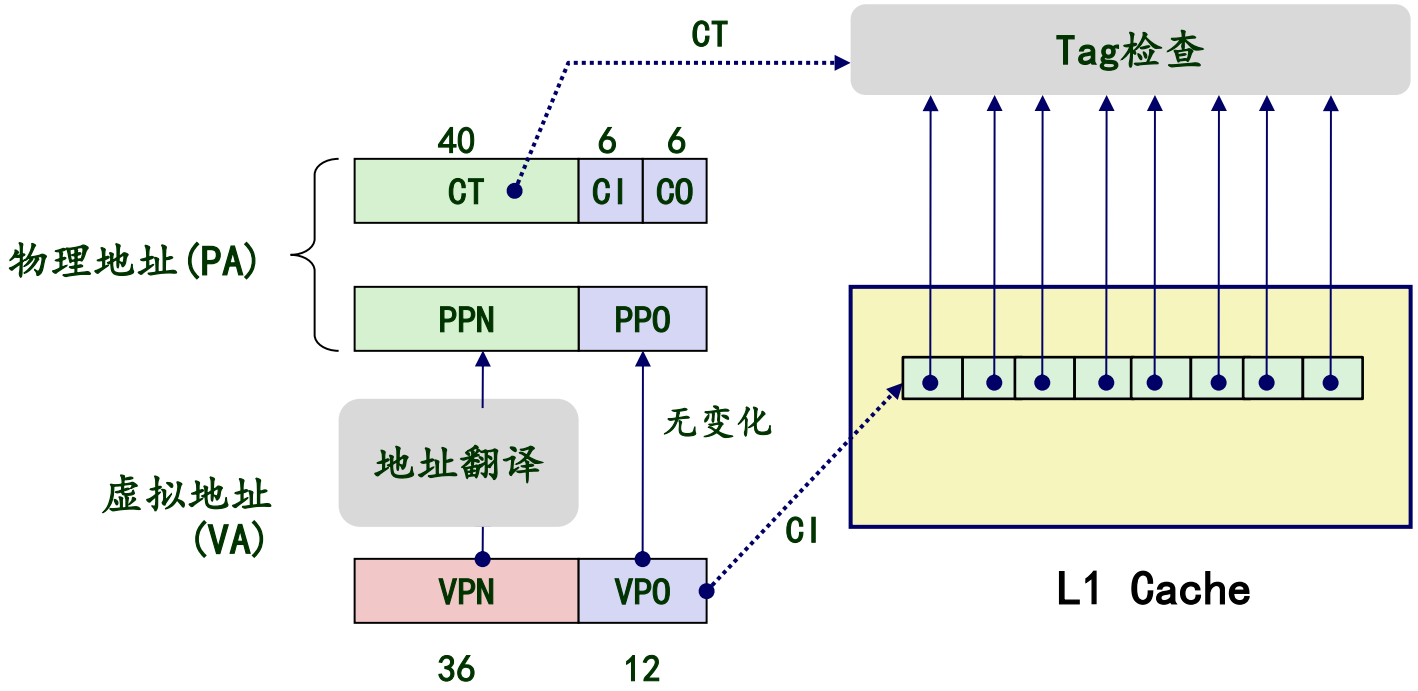
每个条目引用一个 4K 子页面。主要字段说明如下：

- P：1表示页面present；0需由内核判断未映射或未驻留等状态。
- R/W：对子页面的读写权限控制，可设为只读或读写。
- U/S：用户模式或特权（内核）模式的访问权限控制。
- WT：与CD、PAT等共同决定页面的内存类型。
- A：访问位，由硬件在使用该条目时设置，软件可清零。
- D：脏位（Dirty bit），由 MMU 在写操作时自动设置，软件可清零。
- 页面基址：地址位51:12；加页内偏移后才是完整地址。
- XD：禁止或允许从该页面中取指令。

Core i7 页表地址翻译

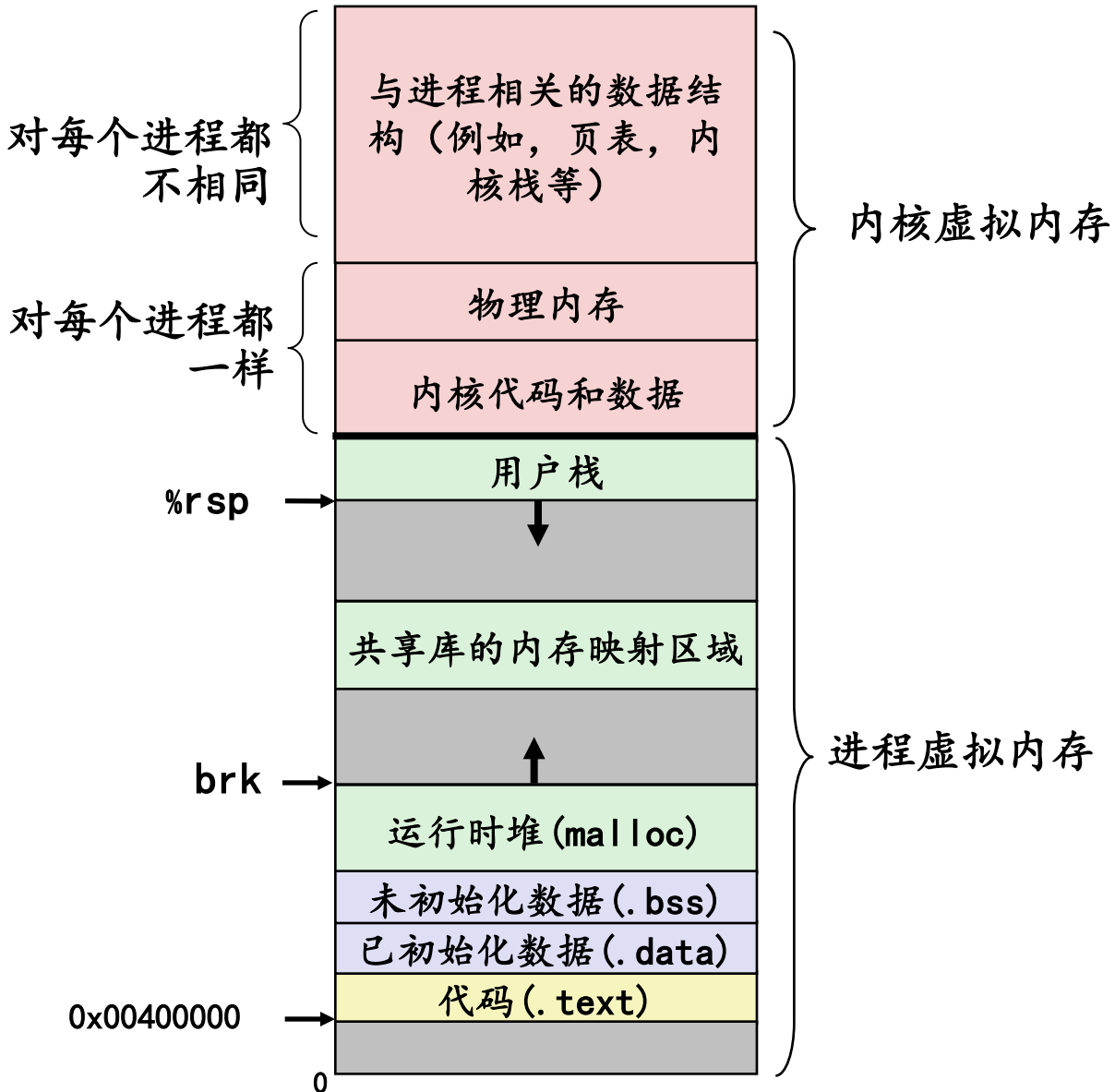


加速 L1访问的技巧

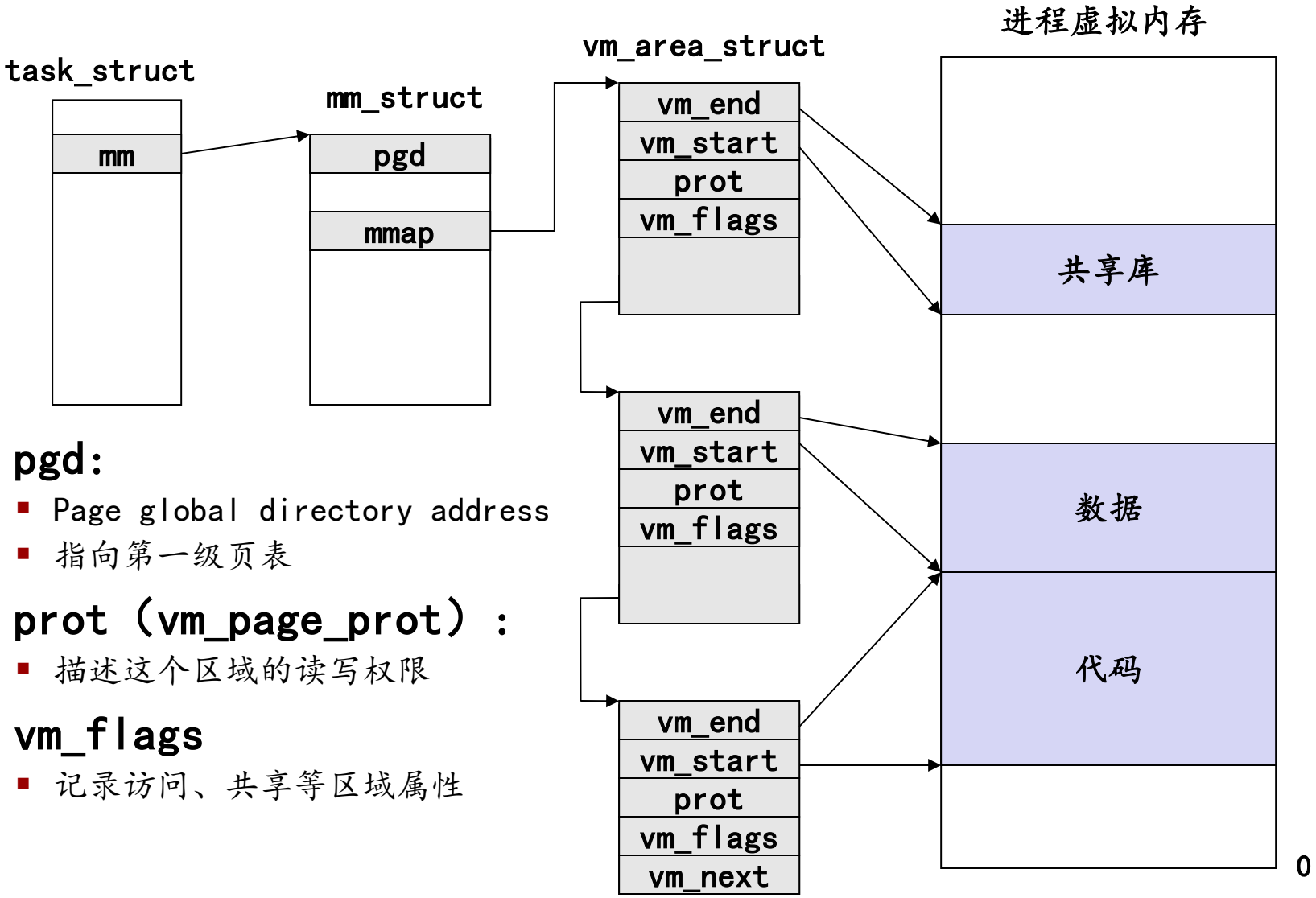


- 观察：
  - 确定在虚拟地址和物理地址中相同的缓存索引（CI）位
  - 在地址转换的同时就可以用这些位索引缓存
  - TLB命中后取得PPN，本例中PPN恰好对应CT。
  - “虚拟索引，物理标记”缓存
  - 缓存需要精心设计大小，以确保上述方法可行。

Linux进程虚拟内存（示意）

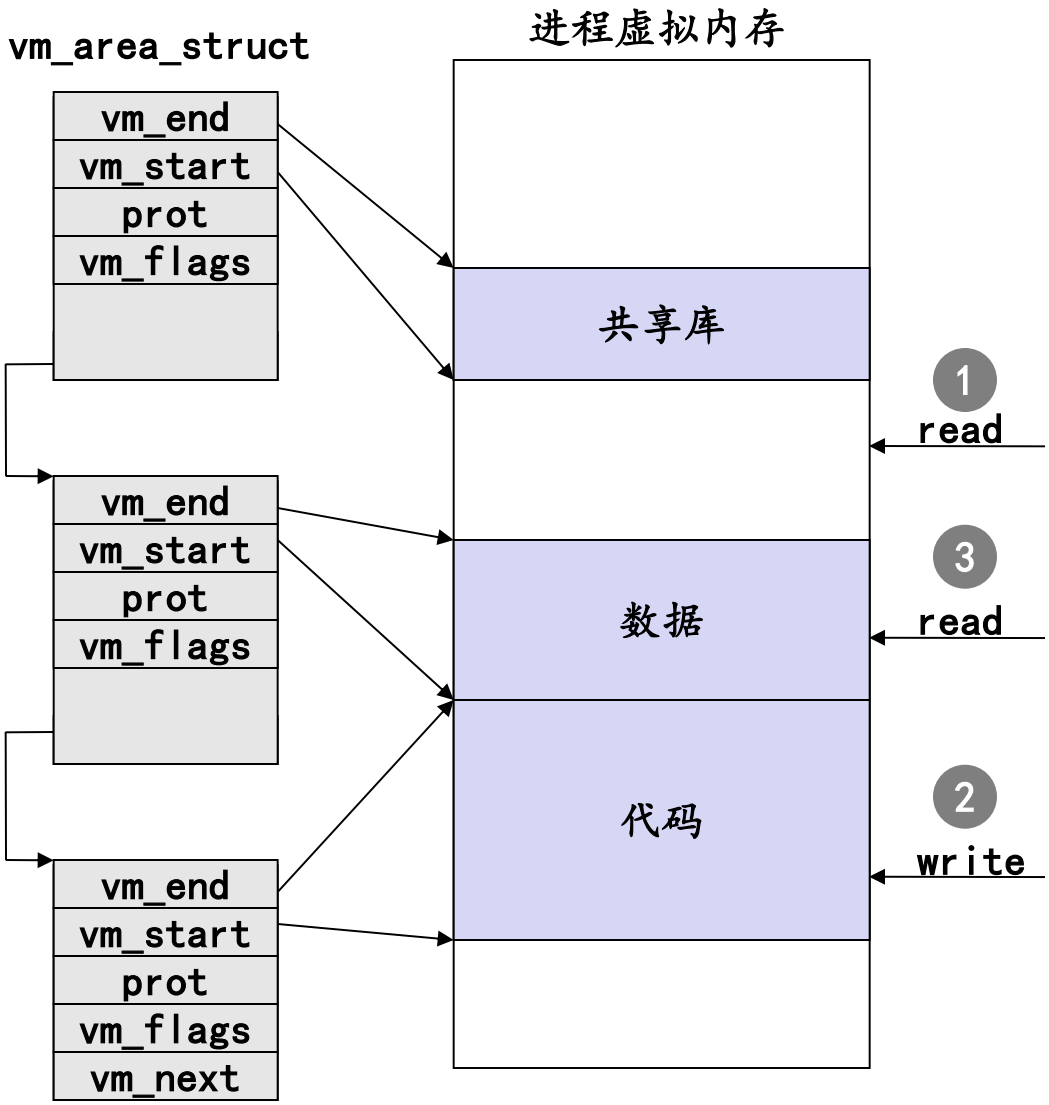


Linux虚拟区域（传统链表示意）



- **pgd:**
  - Page global directory address
  - 指向第一级页表
- **prot（vm_page_prot）：**
  - 描述这个区域的读写权限
- **vm_flags**
  - 记录访问、共享等区域属性

Linux缺页处理



段错误：
访问未映射的非法地址

可恢复的缺页

保护异常：
例如：写入只读且非COW页

主要内容

- 简单内存系统示例
- 案例研究：Core i7/Linux 内存系统
- Linux内存映射

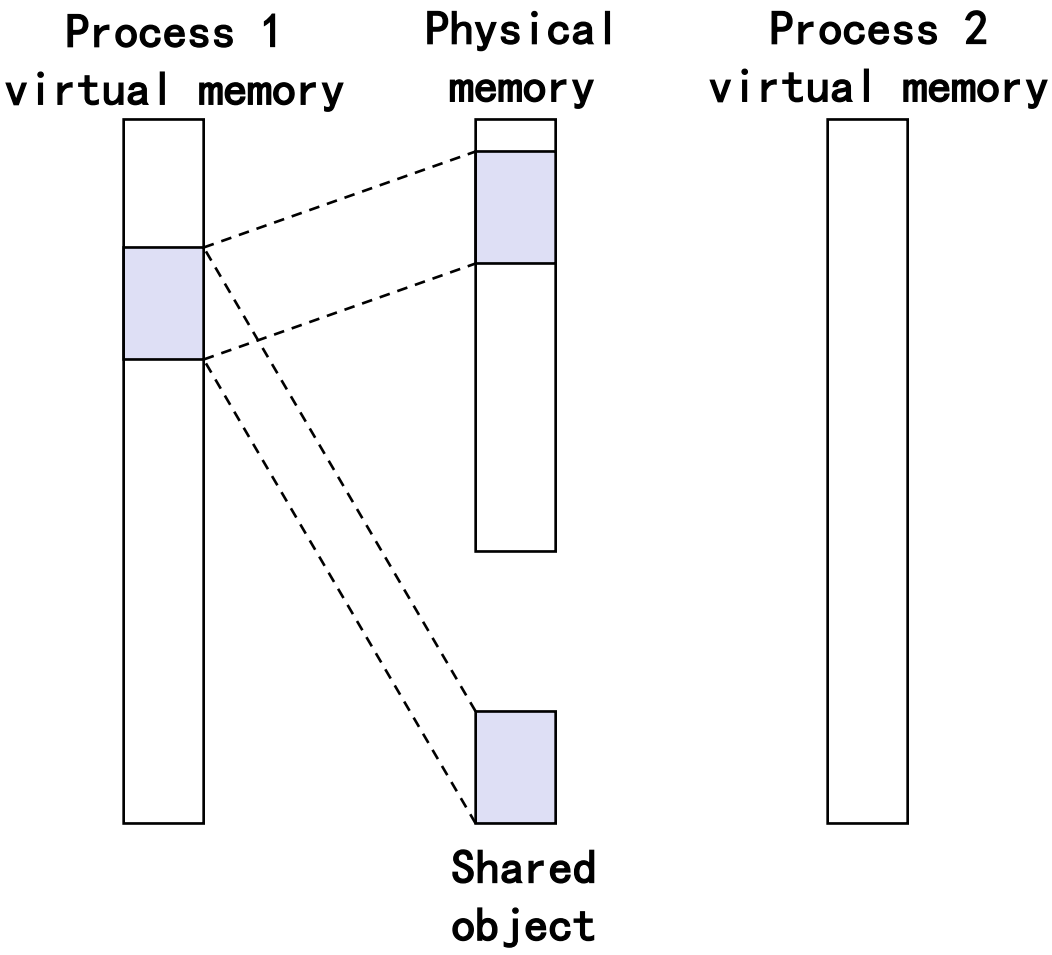
Linux内存映射（Memory Mapping）

- 虚拟内存区域可映射文件，也可建立匿名映射。
  - 这个过程称为 内存映射（memory mapping）。

- 内存区域可以由以下来源支持（也就是初始值来自哪里）：
  - 普通文件（Regular file）：例如磁盘上的可执行文件。
    - 初始页的内容来自文件的某一部分。
  - 私有匿名映射（Anonymous mapping）：初始为零。
    - 按需置零：读时可共享零页，写时再分配私有物理页。
    - 匿名页写入后，需保留修改内容才能回收并恢复。

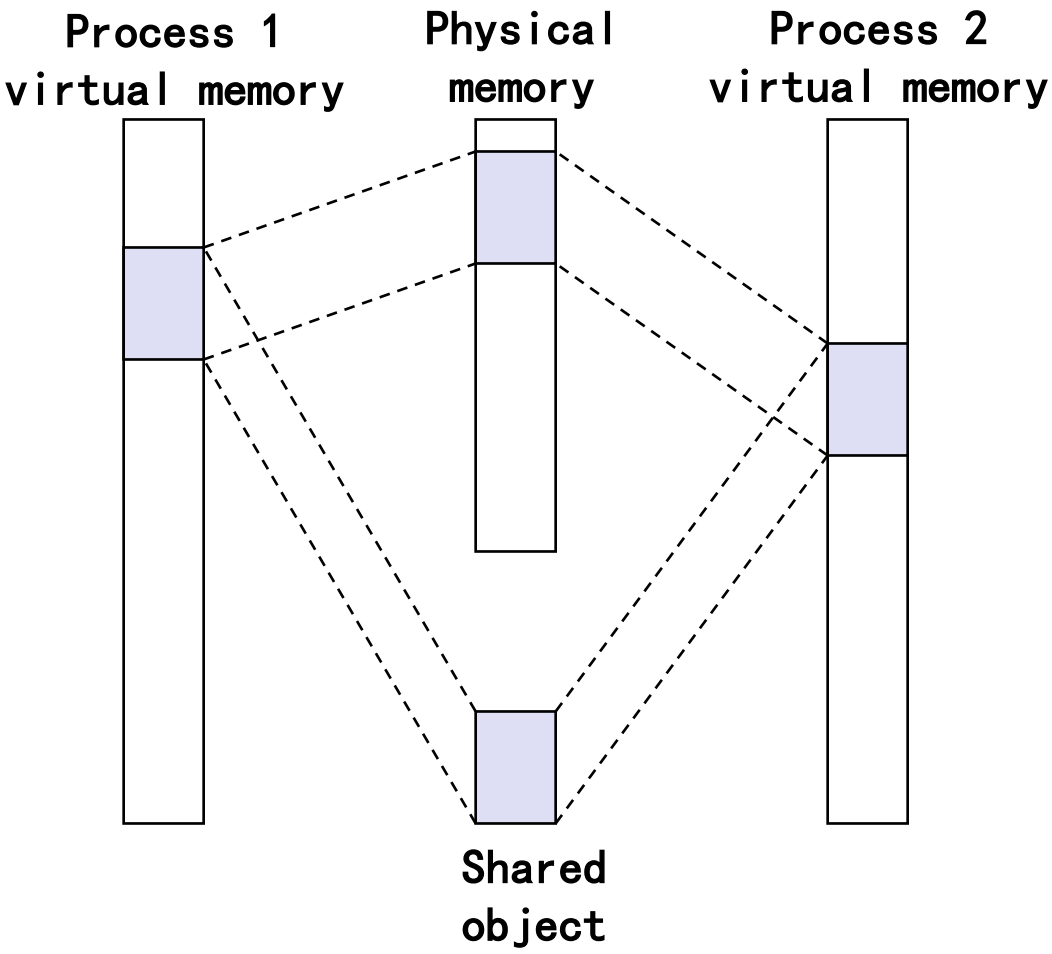
- 匿名页可使用已启用的swap；共享文件脏页可回写文件。

共享对象（Shared Objects）



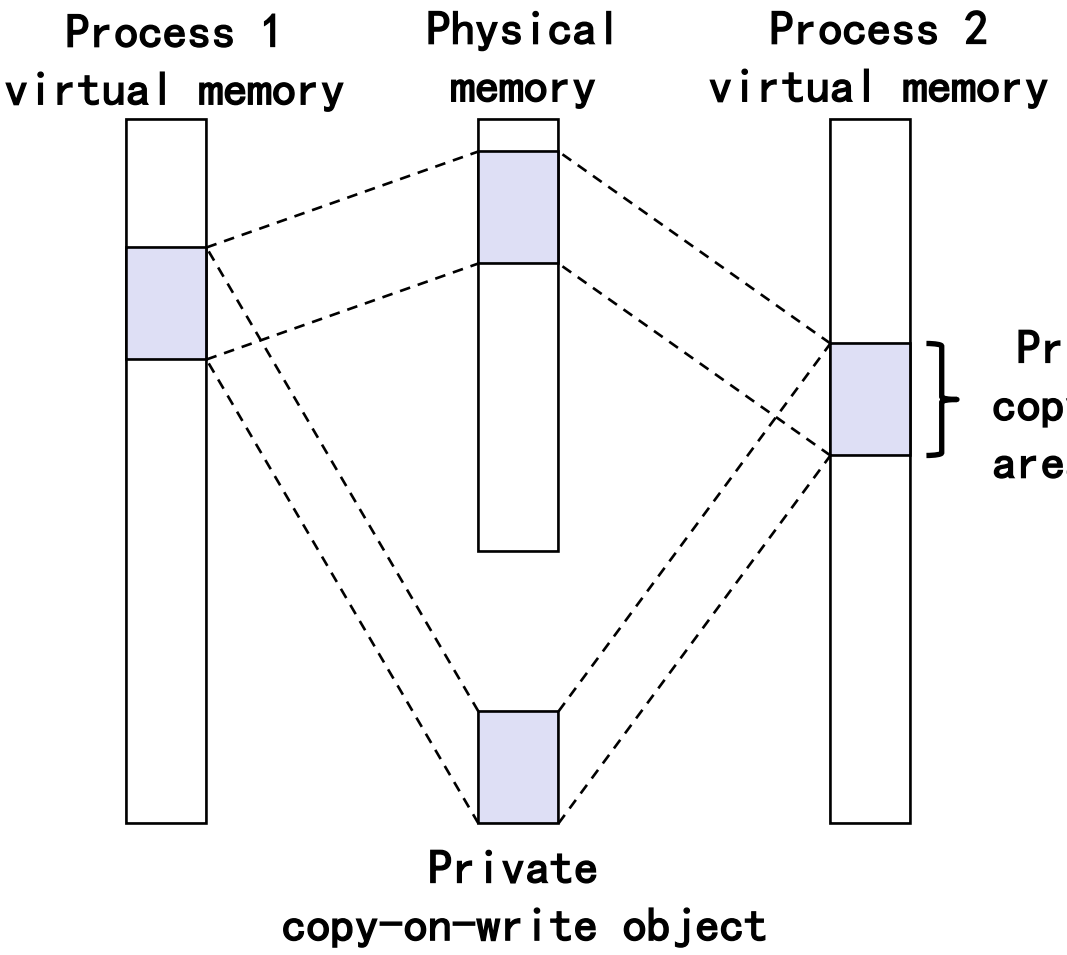
- 进程1通过虚拟内存映射到一块存放了共享对象的物理内存。

共享对象（Shared Objects）



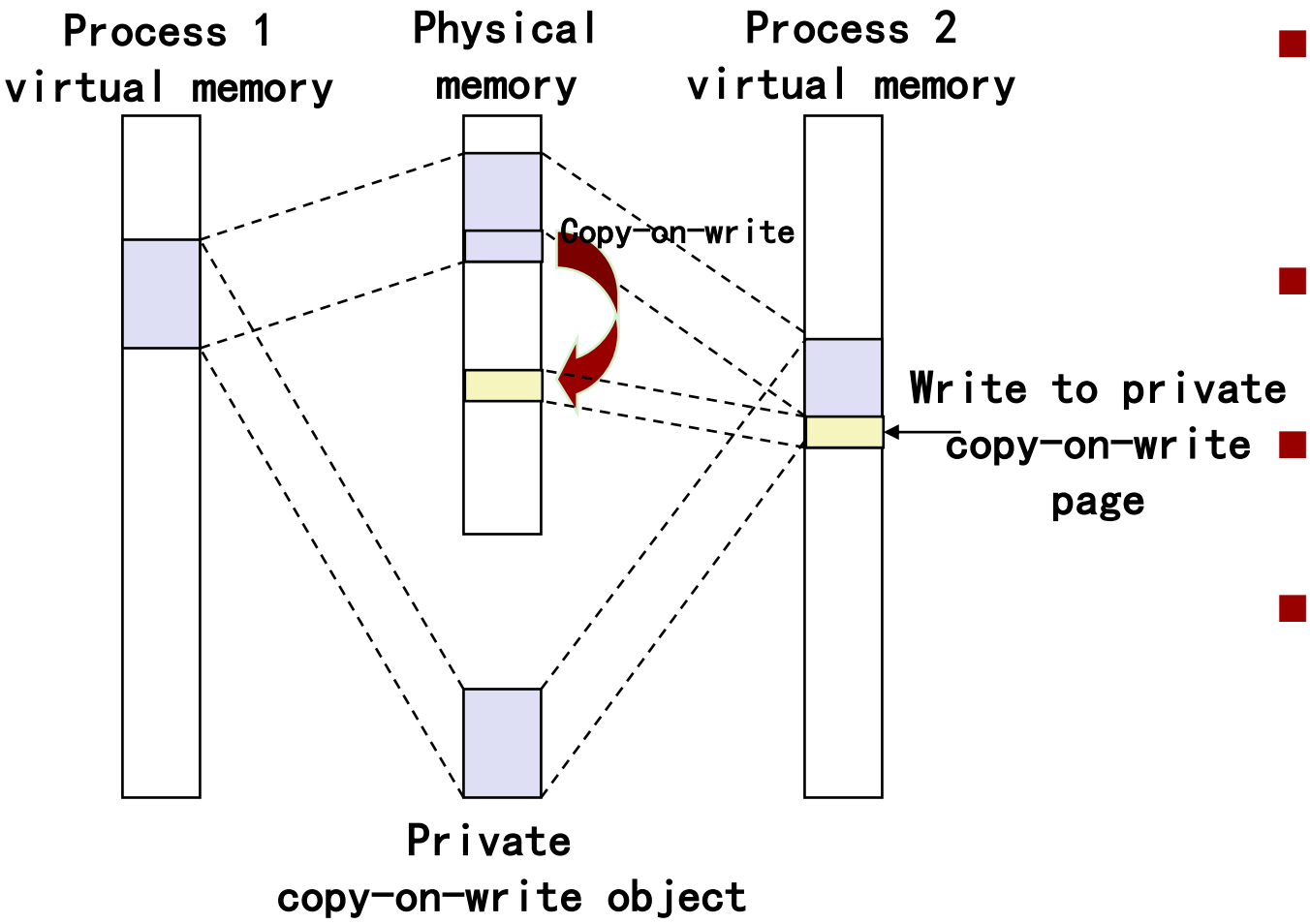
- 进程2也映射到了同一个共享对象。
- 注意：进程1和进程2虽然在虚拟地址空间中访问的是不同的虚拟地址，但实际上它们指向同一块物理内存。

写时复制（Copy-On-Write，COW）



- 两个进程映射一个私有写时复制（COW）对象。
- 标记为私有写时复制的区域
- 私有区域中的 PTE 被标记为只读

写时复制（Copy-On-Write，COW）



- 写入受只读保护的COW页触发页故障。
- 处理程序创建新的读/写页面。
- 处理程序返回后，指令重新启动。
- 复制操作尽可能延迟！

额外：什么是系统调用

- 系统调用是用户程序主动、受控地请求内核服务的接口。
- 用户态受权限限制。read、write、open、fork、execve、mmap 等接口可用于请求内核服务。
- fork()、execve()从哪儿来：
  - 提供者：操作系统内核（Linux 内核）。
  - 语言里的函数：通常由标准库做“包装”（Linux 上多为 glibc）。你在 C 里调 fork()、execve()、read()，其实是调用 glibc 的薄封装，它再把请求递交给内核。
  - 内核里的实现：进入系统调用入口后，分派给对应的内核处理函数。

常见系统调用与库接口

- 进程/线程：fork，vfork，clone，execve，wait，exit，setpriority
- 内存管理：mmap/munmap，brk，mprotect，madvise
- 文件/目录：openat，read，write，close，stat，link/unlink，rename，fsync
- I/O 复用：select，poll，epoll_*
- IPC：pipe，shmget/shmat（SysV），mmap（匿名共享），sem*、msg*
- 网络：socket，bind，listen，accept，connect，send/recv，getsockopt/setsockopt
- 时间/定时器：clock_gettime，nanosleep，timerfd
- 同步/调度：futex，sched_*（如sched_yield）
- 安全/命名空间：setuid/setgid，capset，chroot，unshare，setns，seccomp
- 其他：ioctl（设备控制），getrandom，uname

额外： fork 是什么

- fork 是 Unix/Linux 操作系统里最经典的 系统调用（system call） 之一。
- 它的功能是：创建一个新进程（也叫子进程），几乎是父进程的完整拷贝。名字来自英文的 fork，意思是“分叉”。
- 执行 fork() 后，程序的执行路径就像树枝一样分成两条：一条是父进程继续往下走，一条是子进程开始执行。

额外： exec 是什么

- exec函数族通过execve等系统调用，
  用新程序替换当前进程映像。
- 成功后替换代码、数据、堆和栈，
  从新入口执行；页面按需装入。
- exec不创建新进程，PID不变；
  失败则返回错误，由原程序处理。